

A hobby Xiangqi engine

Building a Xiangqi Engine

From legal moves to evaluation and search, build a chess partner from scratch.

M: chapter map · S: speaker view

A personal starting point

Can a computer help me play?

I calculate slowly and my intuition is unreliable. Computers calculate quickly.

Me

Beginner levels 1–3 on
Tiantian Xiangqi; I often
miss variations.

The computer

It knows no chess, but
repeats calculations
very quickly.

Our task

Break playing chess
into skills and teach
them one by one.

Humans lose track; computers excel at repeated calculations.

Separate the skills of playing chess so we can implement each one.

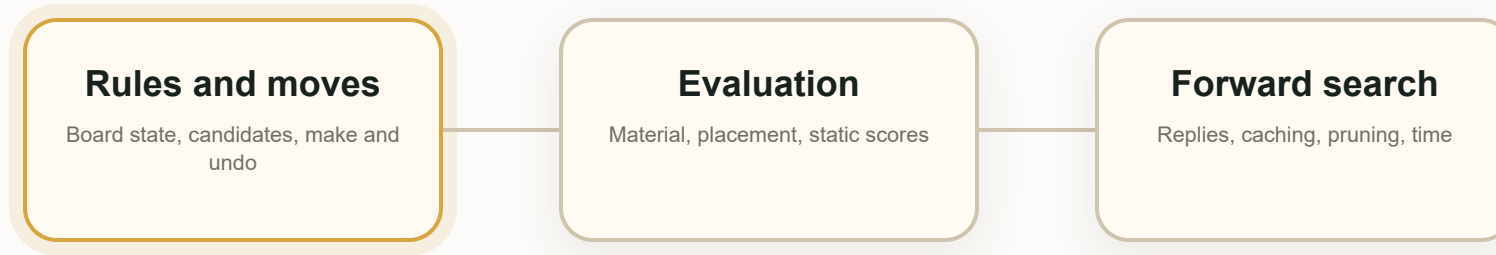
Today we build a Xiangqi partner from scratch.

Start by asking which skills a chess engine needs.

The roadmap

Teach the computer the skills we use

First legal moves, then position evaluation, then forward search.



Move: represent the board, generate candidates, and check legality.

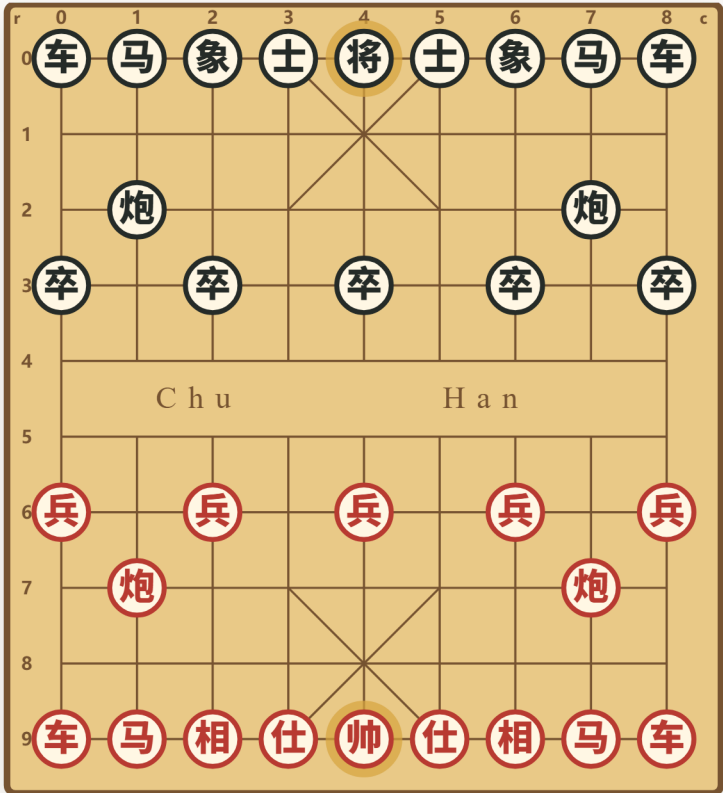
Evaluate: turn a position into a comparable score.

Search: consider replies and remember previously searched positions.

Three stages: moves, evaluation, search.

How does a computer represent the board?

We see a board; the computer needs a readable, writable position.



Initial position, Red to move (row 0 at top, row 9 at bottom)

board[10][9]

One character per square:
uppercase Red, lowercase Black,
dot for empty.

turn

The side to move is part of the
position, even when the board is
identical.

```
board[9][4] = 'K'; // Red king at (9,4)
board[0][4] = 'k'; // Black king at (0,4)
turn = RED;       // Red to move
```

Store every square in a 10×9 character array.

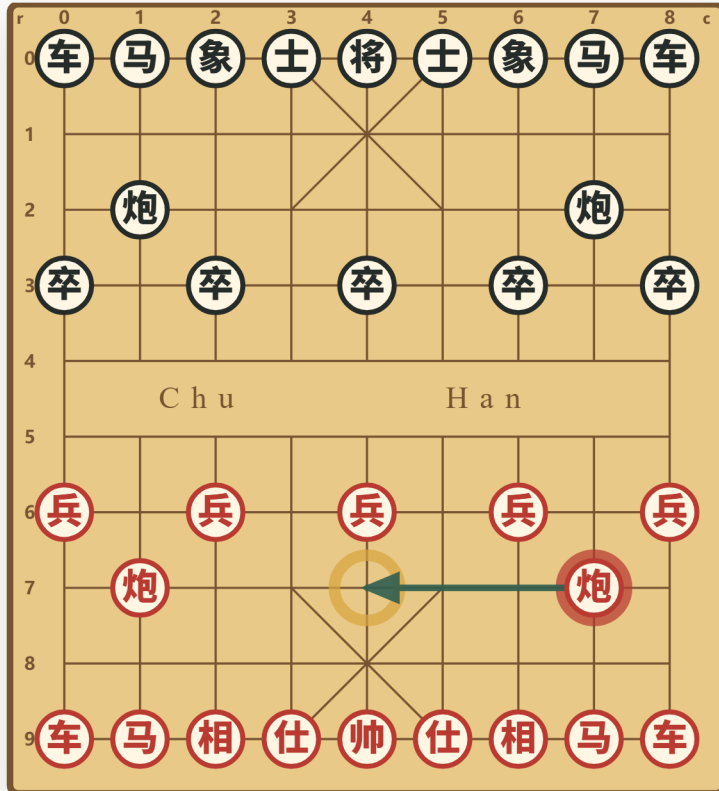
The side to move is an essential part of a position.

Position = 10×9 board + side to move.

Sources: xiangqi_ai.cpp:291 · xiangqi_ai.cpp:390

A move needs a source and a destination

Where does the piece start, and where does it finish?



Red to move: central cannon, (7,7) → (7,4)

```
Move m = {7, 7, 7, 4};  
// (r1,c1) = (7,7) source  
// (r2,c2) = (7,4) destination
```

The source identifies the moving piece.

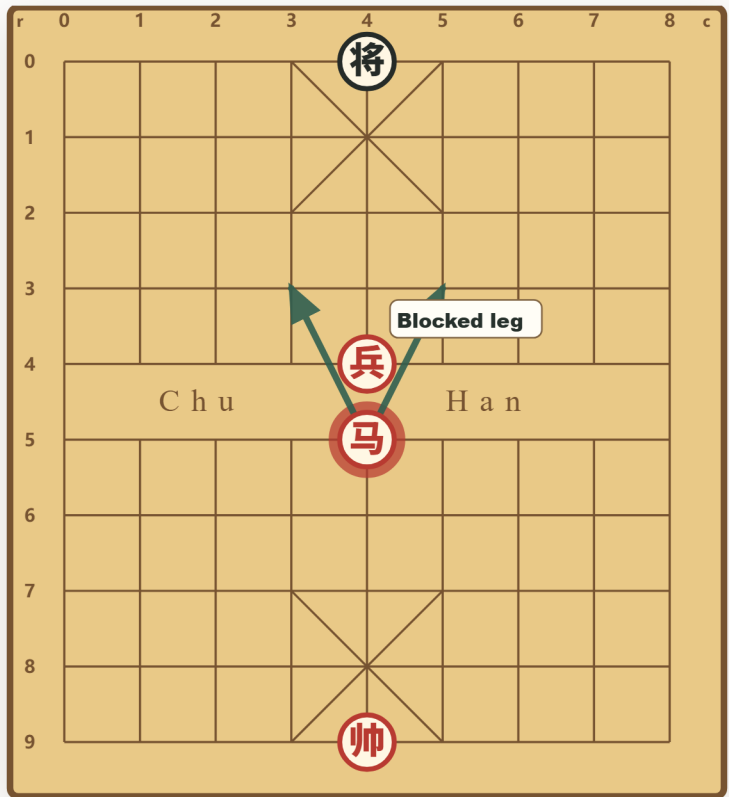
The destination identifies its target square.

Two row/column pairs give four numbers per move.

A move is an operation from one square to another.

Where can this horse move?

Apply the piece's own rules to list possible destinations.



Red horse at (5,4); the pawn at (4,4) blocks its upward leg

- The eight L-shaped destinations are only initial candidates.
- The pawn at (4,4) rules out (3,3) and (3,5).
- Check the remaining targets for board boundaries and friendly pieces.

Candidates depend on movement, boundaries, blockers, and ownership.

Sources: xiangqi_ai.cpp:669

Seven pieces, seven sets of local rules

This layer asks where each piece can move under its own rules.

Piece	Key conditions
Rook	Along a rank or file up to the first blocker
Cannon	Straight quiet moves; exactly one screen for a capture
Horse	L-shaped move; the adjacent leg square must be empty
Elephant	Two diagonal steps; clear eye; cannot cross the river
Advisor	One diagonal step inside the palace
King	One orthogonal step inside the palace; kings must not face
Pawn	Forward only; sideways allowed after crossing the river

- Each piece has its own local reachability conditions.
- A locally valid move may still be illegal for the position.

Piece rules generate candidates; king safety determines legality.

Sources: xiangqi_ai.cpp:677

Iterate only over pieces that exist

The board stores pieces; two-way indices locate pieces and list slots.

**piece_sq[side]
[slot]**

List slot → square;
npieces is the active list
length.

**piece_idx[row]
[column]**

Square → list slot, for
direct updates during
moves and captures.

Piece type

Read the type from
board[row][column];
coordinates distinguish
identical pieces.

Scan the board once at initialization and populate each side's list.

Generate moves by visiting only the first npieces entries.

An ordinary move updates its slot and both square-to-slot references.

Two-way indices support fast iteration and direct updates.

Sources: xiangqi_ai.cpp:771

How do captures avoid holes in the list?

Fill the captured piece's slot with the last entry; undo restores the original slot.

Stage	npieces	Slots 0 / 1 / 2 / 3 (piece names for readability)	Reverse index
Before capture	4	(0,4) K / (2,1) C / (2,7) C / (3,0) P	(2,1)→1; (3,0)→3
Capture cannon in slot 1	3	(0,4) K / (3,0) P / (2,7) C / inactive	(3,0)→1; save original slot 1
Undo: restore last entry	4	(0,4) K / (3,0) P / (2,7) C / (3,0) P	(3,0)→3
Undo: restore cannon	4	(0,4) K / (2,1) C / (2,7) C / (3,0) P	(2,1)→1; (3,0)→3

A memory operation

The pawn's record moves from slot 3 to slot 1; the pawn stays at (3,0).

Boundary case

If the captured piece is last, simply shorten the active list.

Save the captured piece's original slot: 1.

Copy (3,0) from slot 3 into slot 1; update its reverse index to 1.

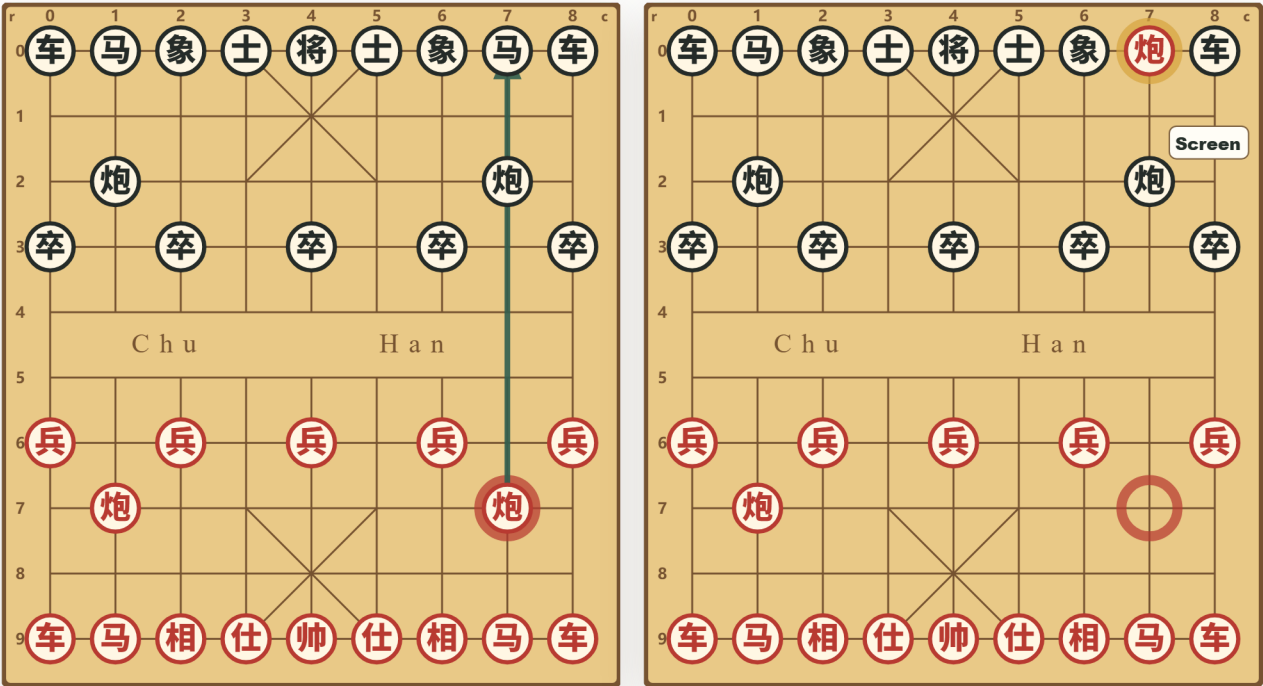
npieces decreases from 4 to 3, keeping iteration contiguous.

Undo moves the replacement back to slot 3 and restores the cannon in slot 1.

Swap removal keeps lists compact; saved slots make undo exact.

Try making a move

A trial move changes the position and remembers the captured piece.



- The cannon moves from (7,7), over the screen at (2,7), to the horse at (0,7).
- The source becomes empty, the target holds the red cannon, and Black moves next.
- make_move returns the captured horse so the move can be undone.

Before: Red to move, right cannon at (7,7)

After: Red cannon captures the horse at (0,7), using (2,7) as a screen; Black to move

make_move updates the position and retains the information needed for undo.

Sources: xiangqi_ai.cpp:485

Why undo a move immediately?

The engine repeatedly explores hypothetical moves.

1. make

Try a candidate, obtain a new position, and remember any capture.

2. inspect

Check legality; later, search further from this position.

3. undo

Restore source, target, capture, turn, and all incremental state.

4. next

Return to exactly the original state before trying another candidate.

Making a move enters a hypothetical future.

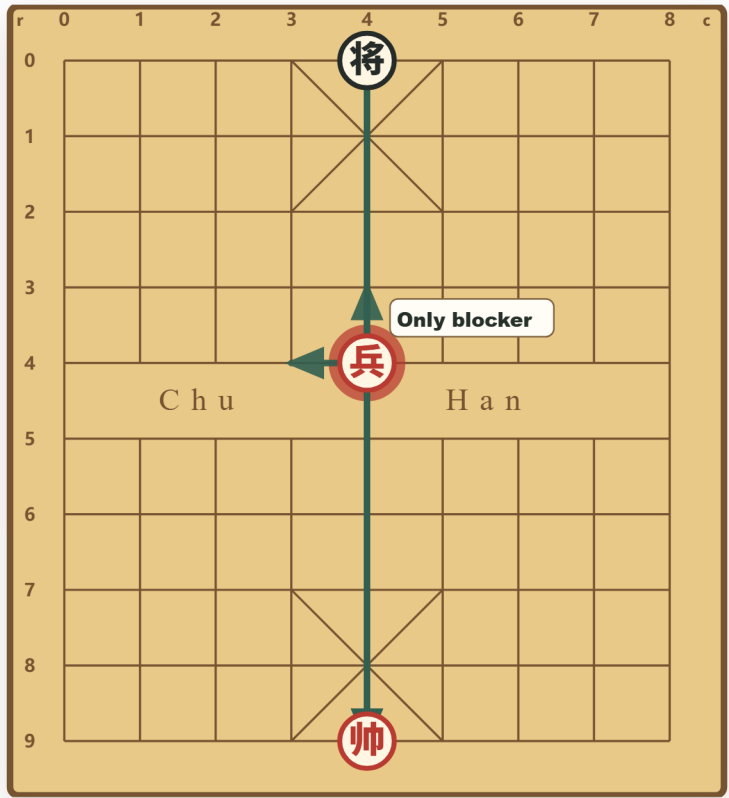
Inspect that future for legality or further variations.

Undo restores every state component before the next candidate.

The basic exploration loop is **make** → **inspect** → **undo**.

A piece can move there. Is the move legal?

A sideways pawn move may obey its local rule but expose the two kings.



Red to move: (4,4)→(4,3) exposes the facing kings

- After crossing the river, the pawn may locally move (4,4)→(4,3).
- Moving sideways leaves no piece between the kings.
- (4,4)→(4,3) is pseudo-legal but illegal; (4,4)→(3,4) keeps the file blocked and is legal.

Local piece movement alone does not establish position legality.

Sources: xiangqi_ai.cpp:735 · xiangqi_ai.cpp:793

A simple, reliable legality check

Create the candidate position, then check the moving side's king.

```
const bool red_mover = (turn == 0); // Save the moving side
std::vector<Move> legal_moves;
for (Move m : pseudo_moves) {
    char captured = make_move(m);
    bool legal = !is_in_check(red_mover);
    undo_move(m, captured);
    if (legal) legal_moves.push_back(m);
}
```

Make

Put the board into the candidate position.

Check

Test enemy rook, cannon, horse, pawn, and king attacks on our king.

Always undo

Return to the same starting position whether the move is legal or not.

Make the move to create the candidate position.

Check whether our king is under attack.

Always undo; retain only candidates that leave our king safe.

Pseudo-legal moves become legal after make → check → undo.

Why cache the king positions?

Every trial move checks king safety; repeatedly locating the king wastes work.

Find it again

Scan 90 squares, then check attacks.

Keep it cached

king_pos[2] stores both kings; update on king moves, captures, and undo.

The same pattern

Piece lists also retain answers to frequent queries in state.

Scanning 90 squares before every check repeats the same work.

king_pos changes only when the king position changes.

Piece lists and king positions are indices for frequent queries.

Maintain frequent-query answers incrementally through make/undo.

Sources: xiangqi_ai.cpp:299 · xiangqi_ai.cpp:490

No legal moves means the game is over

Checkmate and stalemate share one question: does the moving side have any legal move?

In check + no legal move

Checkmate: the side to move loses.

Not in check + no legal move

Stalemate: the side to move also loses in Xiangqi.

One engine interface

Generate and filter candidates; zero legal moves means a loss.

In check with no legal move is checkmate.

Not in check with no legal move is stalemate.

Both are losses for the side to move in Xiangqi.

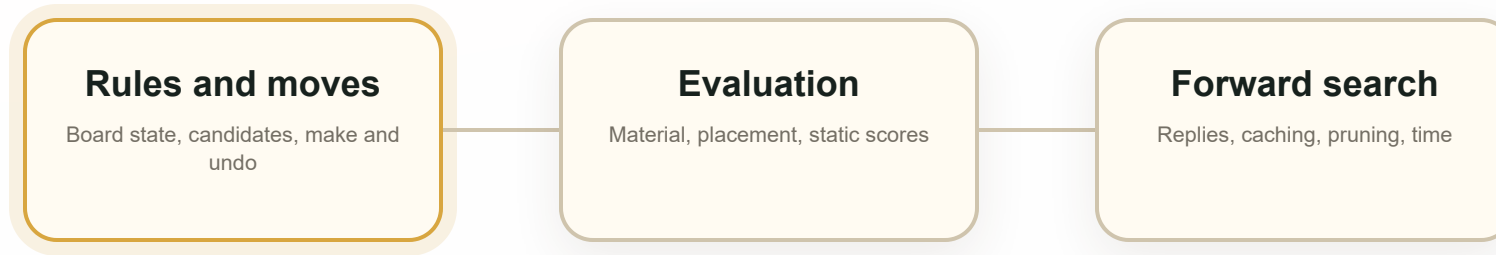
Zero legal moves ends the game; both mate and stalemate are losses.

Sources: xiangqi_ai.cpp:1250 · xiangqi_ai.cpp:1516 · webapp.py:115

Stage 1 complete

We have built a world with rules

We can now try every legal option. Next, decide which options are better.



Represent positions and propose candidates using piece rules.

Filter legal moves through make, check, and undo.

It cannot choose well yet, but it can move legally.

First skill unlocked: legal moves.

Next: position evaluation

All are legal. Which is better?

Legality rejects impossible moves; the remaining moves still need comparison.

Candidate A

Legal, but may lose material or worsen the position.

Candidate B

Also legal, but may gain material or improve placement.

The next question

How can position quality become a comparable score?

The rules layer retains every legal candidate.

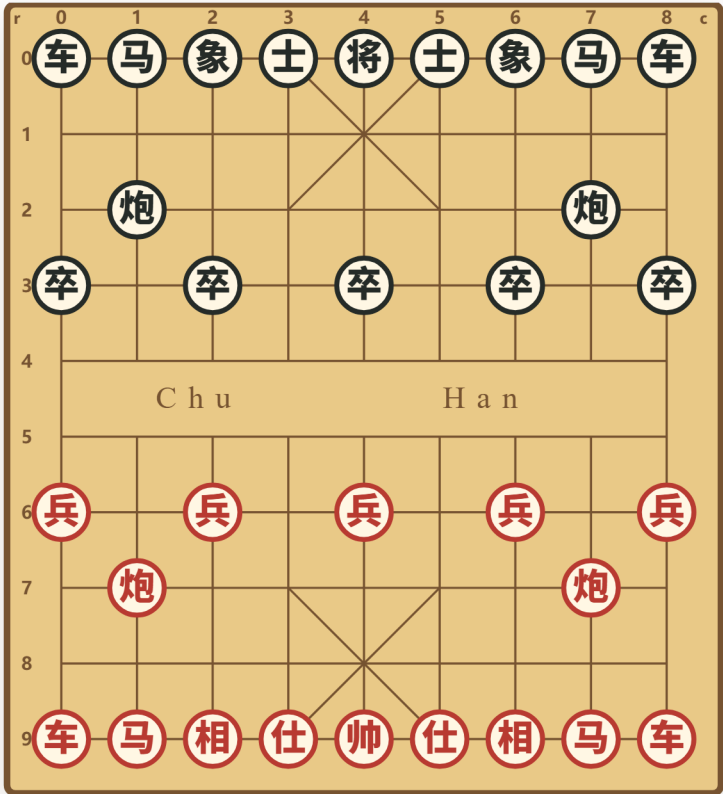
Legal candidates can differ greatly in quality.

An evaluation function lets the program compare positions.

Rules establish legality; evaluation distinguishes quality.

We can move. Which move should we choose?

There may be dozens of legal moves. We need a language for comparing future positions.



Initial position · Red to move

Evaluation gives search leaves a common scale.

Input

A specific position

Output

A static evaluation score

A position where the search stops expanding is a leaf.

At a leaf, how do we decide who is better?

Turn intuition into a comparable number.

The first measure

Basic intuition: how much material remains?

Assign each piece type a base value, then sum for Red and Black.

Piece	King	Rook	Horse	Cannon	Advisor / elephant	Pawn
Project base values	10000	1000	450	450	120	100

```
score = sum(Red piece values) - sum(Black piece values);
```

Red material total – Black material total

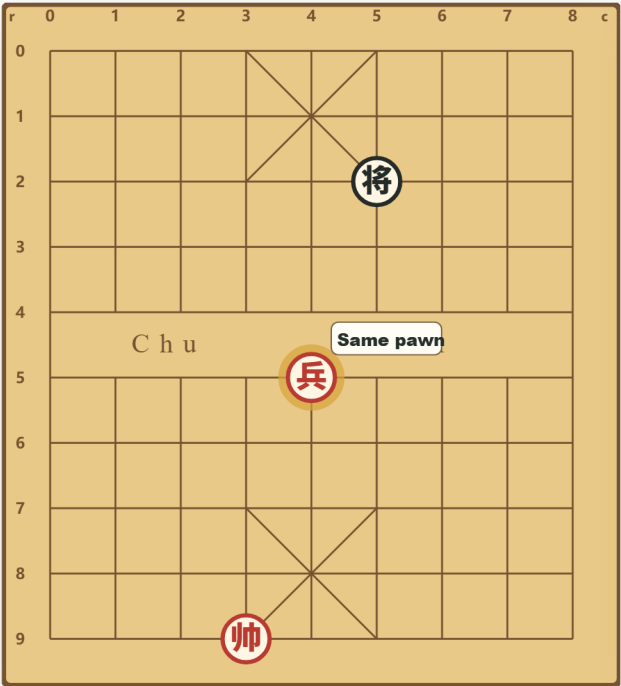
Winning a rook is usually worth more than winning a pawn.

Material describes what remains, but not where it stands.

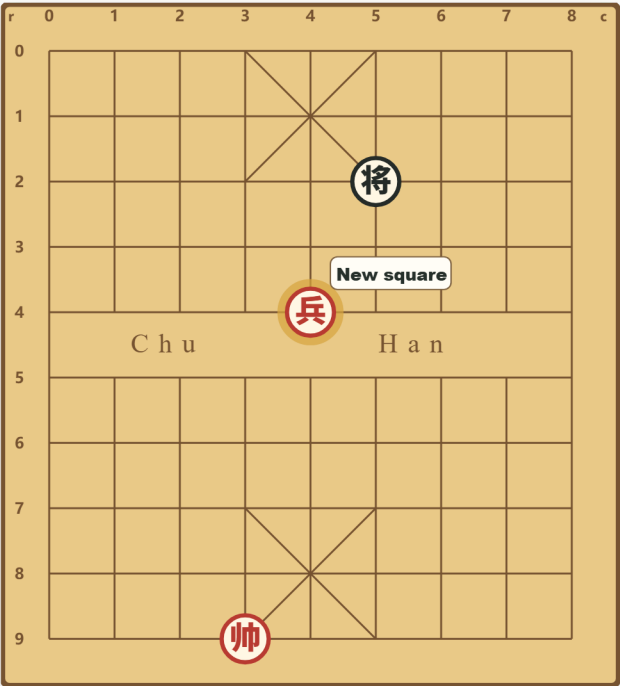
A first counterexample

Same pieces, same position quality?

Material is identical; the pawn on the right has crossed the river, yet material-only scores are equal.



Red to move · Central pawn before crossing



Red to move · Central pawn after crossing

- Material is unchanged.
- After crossing, the pawn also gains sideways movement.
- Can evaluation account for placement?

Equal material can deserve different scores: add position.

A table of squares for each piece

PST: piece type $\times$ square $\rightarrow$ a predefined value.

r	0	1	2	3	4	5	6	7	8	c
0	9	9	9	11	13	11	9	9	9	
1	39	49	69	84	89	84	69	49	39	
2	39	49	64	74	74	64	49	39		
3	39	46	54	59	61	59	54	46	39	
4	29	37	41	54	59	54	41	37	29	
5	7	0	13	0	16	0	13	0	7	
6	7	0	7	0	15	0	7	0	7	
7	0	0	0	0	0	0	0	0	0	
8	0	0	0	0	0	0	0	0	0	
9	0	0	0	0	0	0	0	0	0	

Red pawn PST · Red perspective · 90 entries

A lookup takes $O(1)$.

Black uses the same table with ranks mirrored.

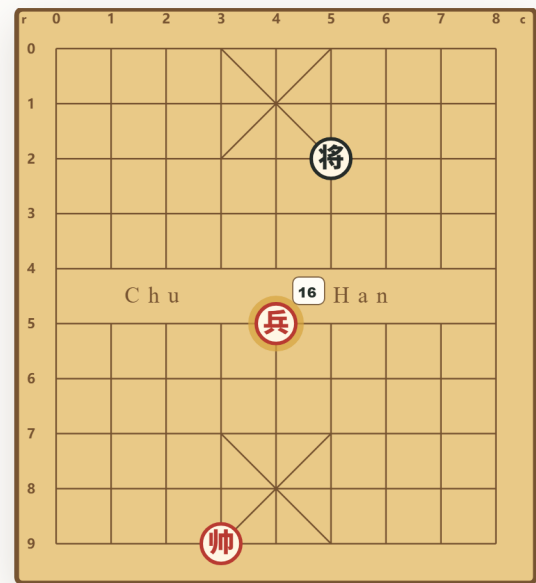
Zeros on the rear three ranks mark unreachable pawn squares, not worthless legal squares.

One PST lookup combines piece type and location.

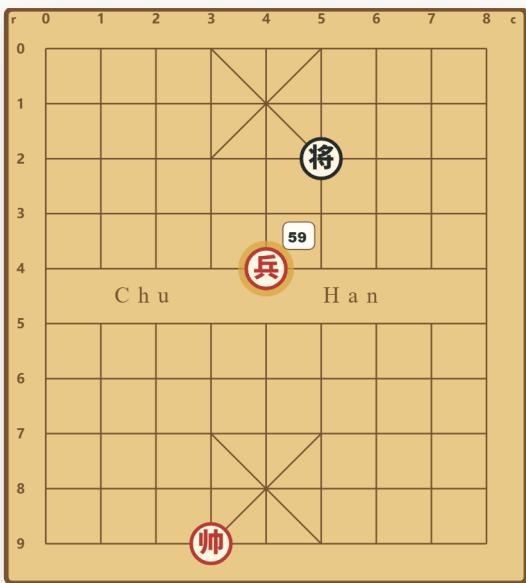
PST suggests; search checks consequences

What does 16 → 59 → 89 → 13 express?

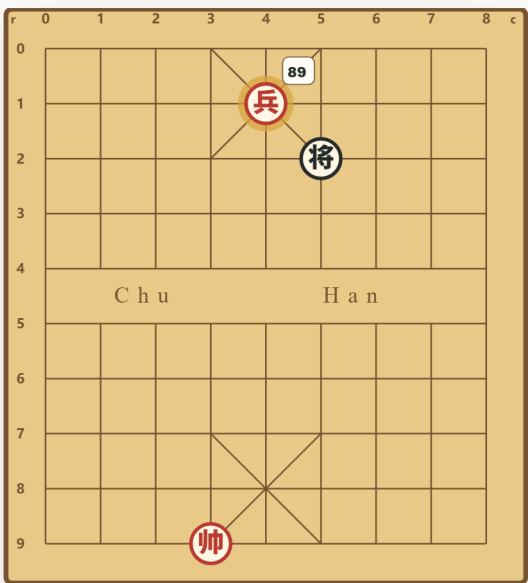
Four diagrams show one pawn's table value at four points on a file.



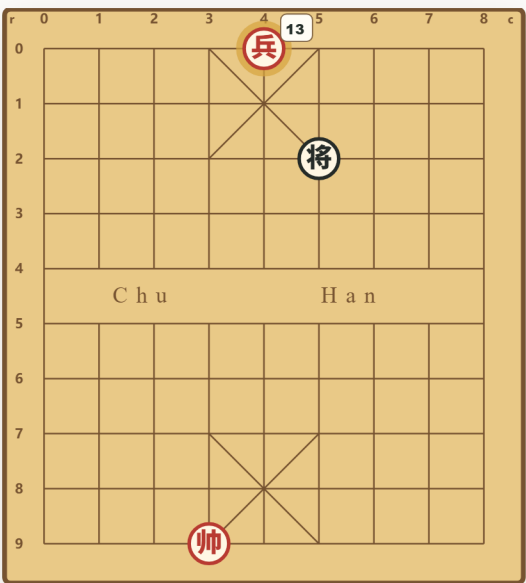
Before crossing · 16



Just crossed · 59



Penultimate rank · 89



Back rank · 13

The table favors advanced pawns on the penultimate rank.

The back-rank value 13 is a static preference; search determines tactical value.

PST encodes heuristics; the position and search determine the outcome.

Wins and losses are not ordinary scores

Material and position compare ongoing games; decisive outcomes use scores near ± 30000 .

Ongoing position

Material and square values measure relative positional quality.

Red wins

Black king absent, or
Black has no legal move: return near +30000.

Black wins

Red king absent, or
Red has no legal move: return near -30000.

Why separate them?

A decisive outcome must outweigh ordinary material or positional gains.

SCORE_INF is 30000 in this project.

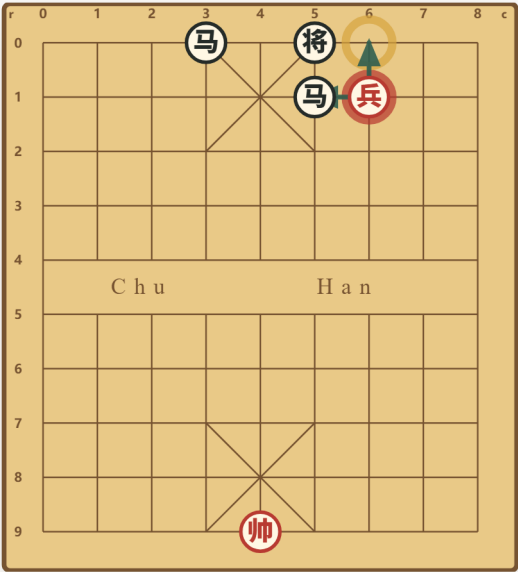
A missing king or no legal move establishes a decisive result.

Terminal scores greatly exceed ordinary evaluation magnitudes.

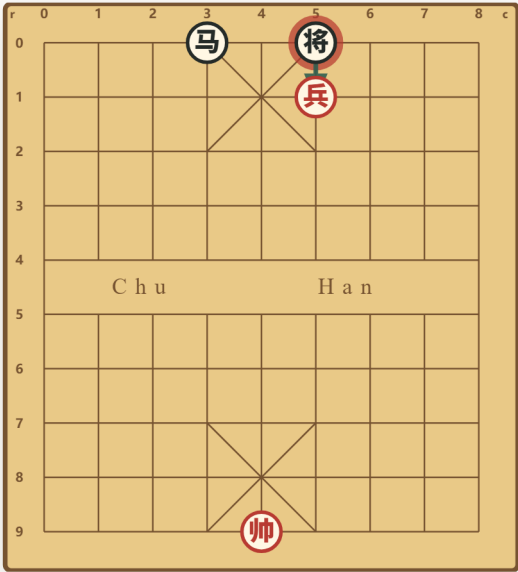
Static evaluation compares ongoing positions; terminal scores encode decisive outcomes.

Why advance the pawn for a static loss of 60?

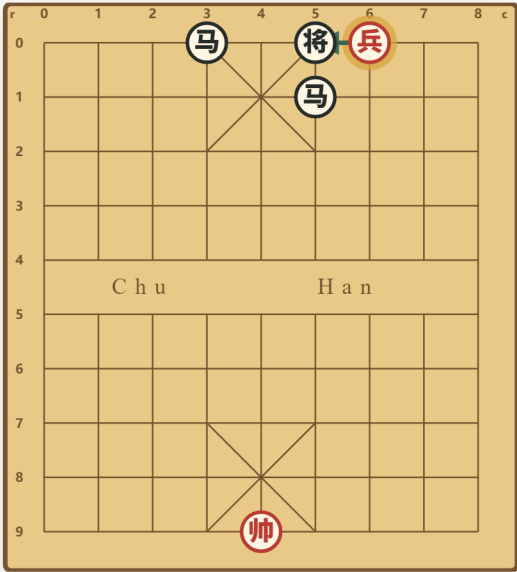
Capturing the horse looks better immediately; exploring replies reveals a winning alternative.



Before · Red to move · Two candidates



Pawn captures horse sideways; Black king can recapture on (1,5)



Pawn advances; Black cannot escape check: mate in one

Immediate capture gain

Black horse total 102;
pawn rises from 69 to 84:
static gain +117.

Static advance cost

Pawn PST falls from 69 to 9:
-60.

After searching

Advancing delivers mate,
returning a score near
+30000.

Material gain usually exceeds a small
positional improvement, making the
capture attractive.

The advance loses 60 statically but
leaves Black no escape from check.

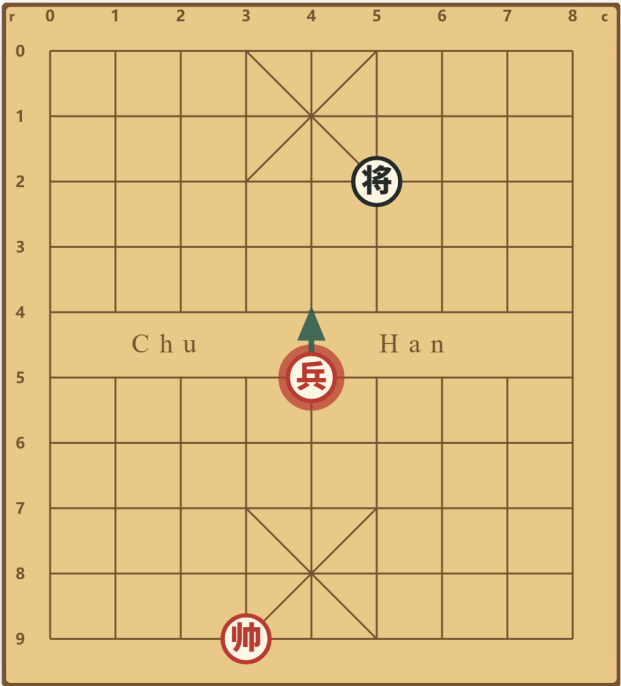
Black's king recaptures the pawn;
search removes it and continues the
exchange line.

Search follows exchanges; their net gain is still below the alternative's decisive win.

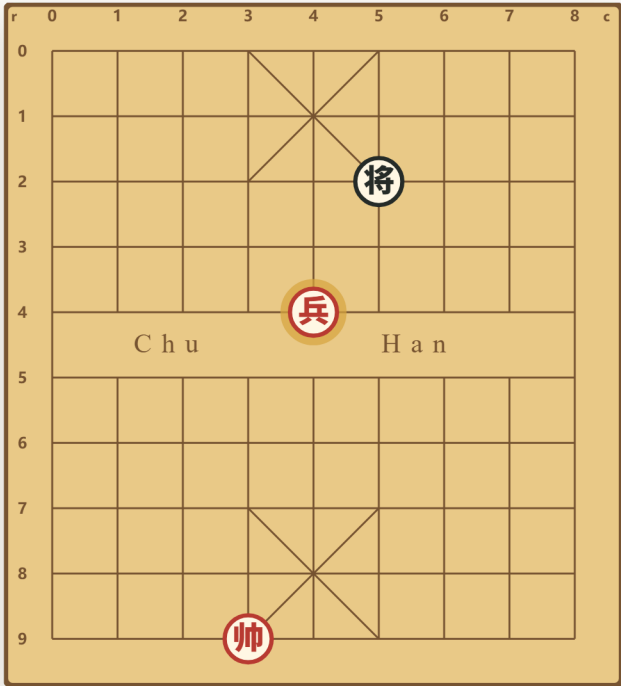
Avoid scanning all 90 squares

A quiet advance changes only two values

The same legal central-pawn advance, with an empty target.



Before advance · Pawn value 16



After advance · Pawn value 59

$\Delta \text{score} = -16 + 59 = +43$

$\Delta \text{score} = -16 + 59 = +43$

Subtract the old square value and add the new one.

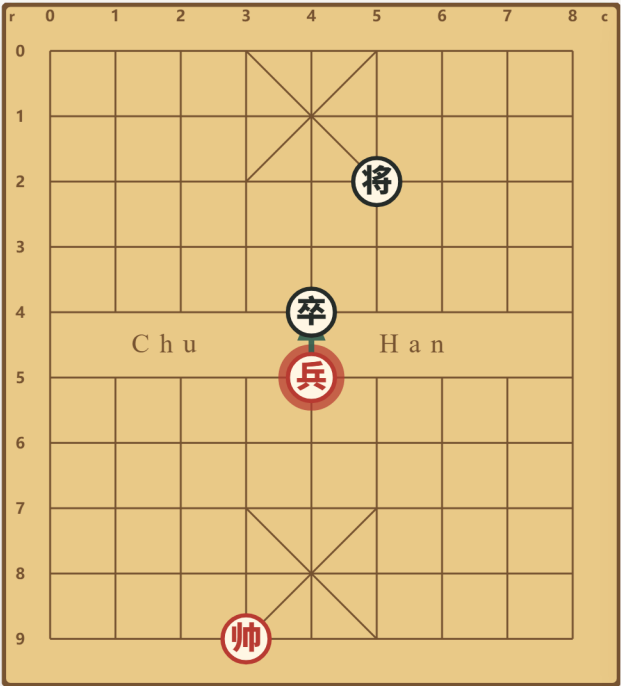
Undo applies the exact inverse.

Maintain evaluation with moves instead of rescanning each search node.

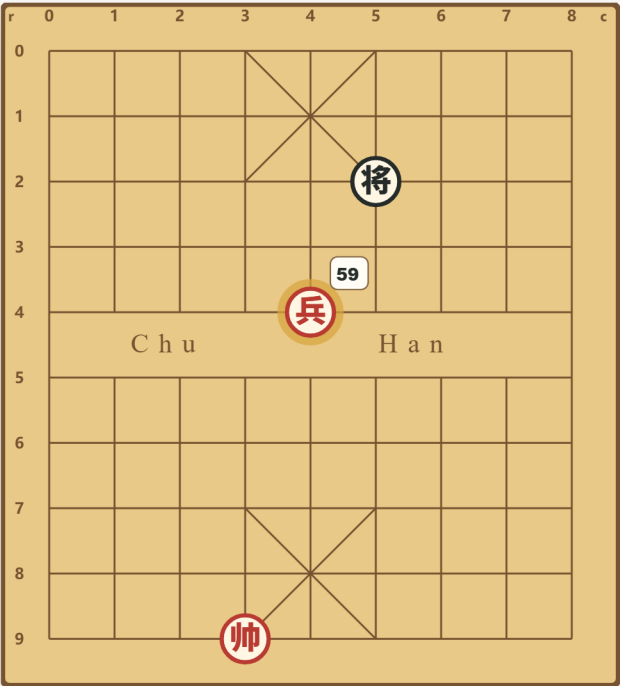
Incremental scoring

If the target is occupied, remove its value too

The same pawn advance, now capturing a black pawn.



Before · Black pawn ahead



After · Black pawn removed

$\Delta\text{score} = -16 - (-16) + 59 = +59$

$\Delta\text{score} = -16 - (-16) + 59 = +59$

Remove the old red-pawn value: -16.

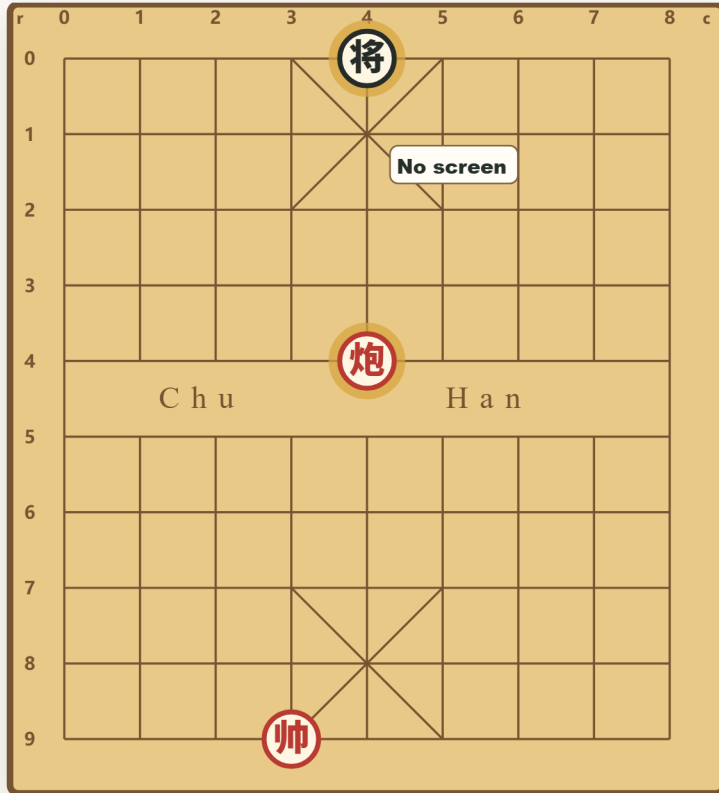
Remove Black's contribution: $-(-16)$; add the new red-pawn value: +59.

Moves, captures, and undo update only affected pieces.

A second counterexample

Independent square values miss relationships

Adding separate piece scores loses interactions between pieces.



Open-file cannon · No screen, so no current check

Defensive structure

Mutual protection by advisors and elephants is a relationship, not one square value.

Open-file cannon

Its value depends on the line and occupancy between cannon and enemy king.

Pins / control

Moving one piece can change another piece's legal space.

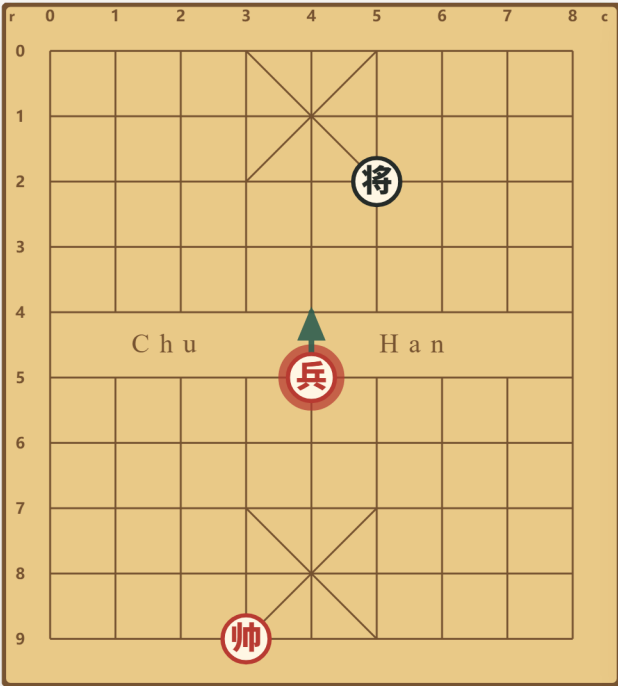
PST asks only: what piece, and where?

Relational features ask: with whom, behind which blocker, controlling what together?

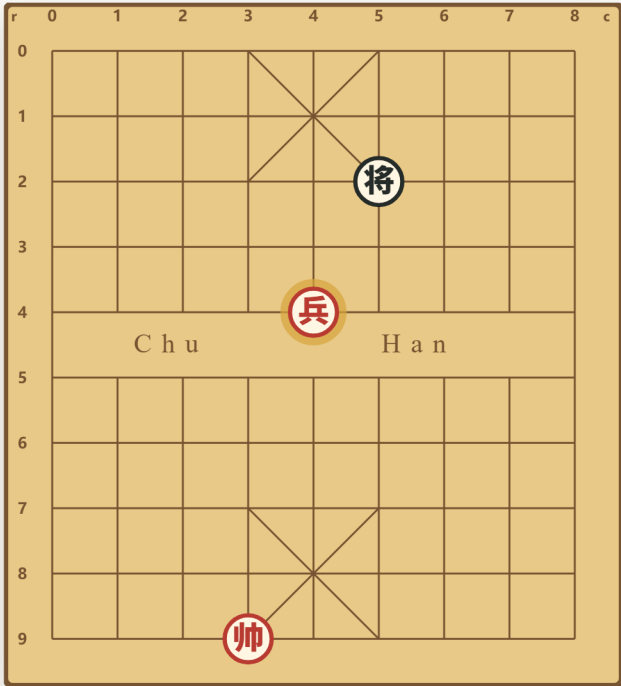
Single-piece features are cheap; richer relationships require more computation.

Model relationships while updating few features

NNUE extends the incremental PST idea from one number to a vector of intermediate features.



Before · Pawn has not crossed



After advance · Pawn has crossed

PST

$P[\text{pawn}, \text{square}]$ is a scalar: one number.

NNUE

$W[\text{pawn}, \text{square}]$ is a first-layer weight column: a vector.

PST: $S' = S - P[\text{pawn}, \text{old}] + P[\text{pawn}, \text{new}]$

NNUE: $A' = A - W[\text{pawn}, \text{old}] + W[\text{pawn}, \text{new}]$

Only a few piece-on-square features are active.

Both PST and NNUE subtract the old feature and add the new one.

PST updates a scalar; NNUE updates a vector. Both touch only changed features.

Sources: xiangqi_ai.cpp:791

Cache the first-layer sum in an accumulator

A stores the sum of contributions from all active board features.

```
Accumulator      A = [10, 20]
Old contribution Wold = [2, 3]
New contribution Wnew = [5, 4]
```

```
A' = A - Wold + Wnew
    = [10,20] - [2,3] + [5,4]
    = [13,21]
```

Two-dimensional teaching example, not actual network weights.

Weight vector W

The learned contribution of one piece-square feature.

Accumulator A

Sum all active feature columns and retain the result.

The pawn leaves its old square: subtract that column.

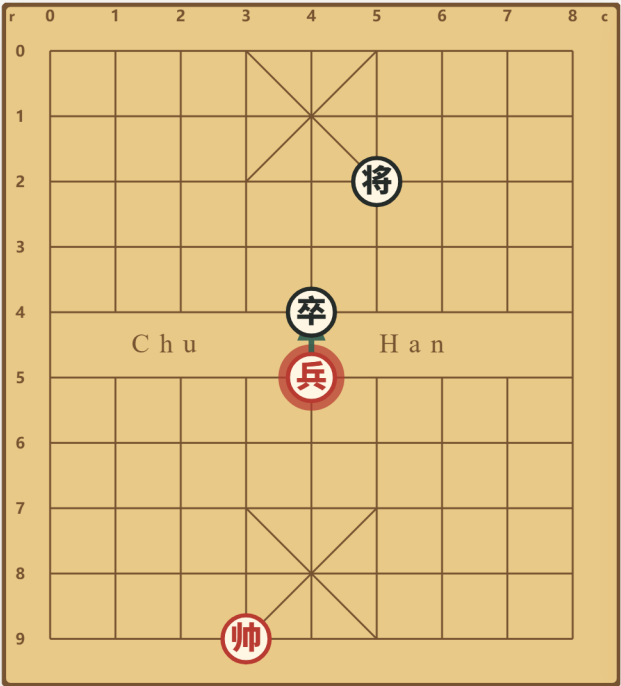
It appears on the new square: add that column.

Other pieces' contributions remain cached in A.

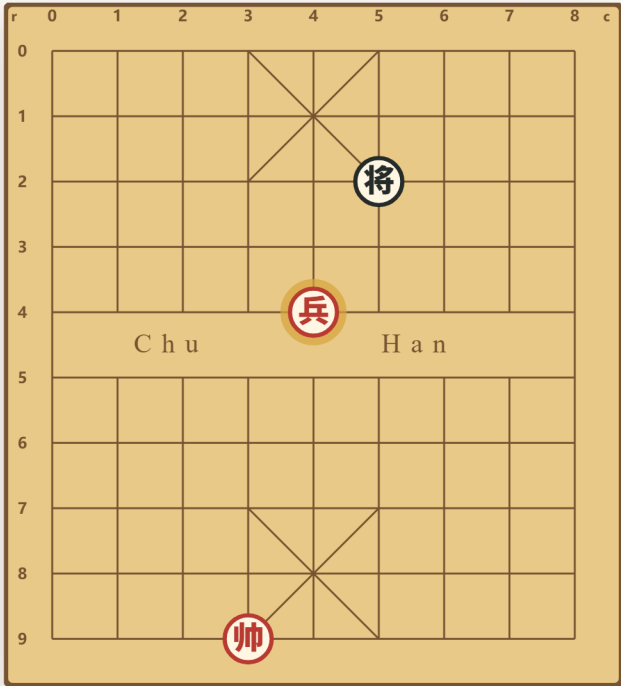
The accumulator replaces full recomputation with subtract-old, add-new.

Captures also subtract the captured vector

The pawn advances again, this time onto a black pawn.



Before · Black pawn ahead



After · Black pawn removed

$$A' = A$$

- $W[\text{red pawn, old square}]$
- $W[\text{black pawn, captured square}]$
- + $W[\text{red pawn, new square}]$

Moving piece: subtract its old feature and add its new feature.

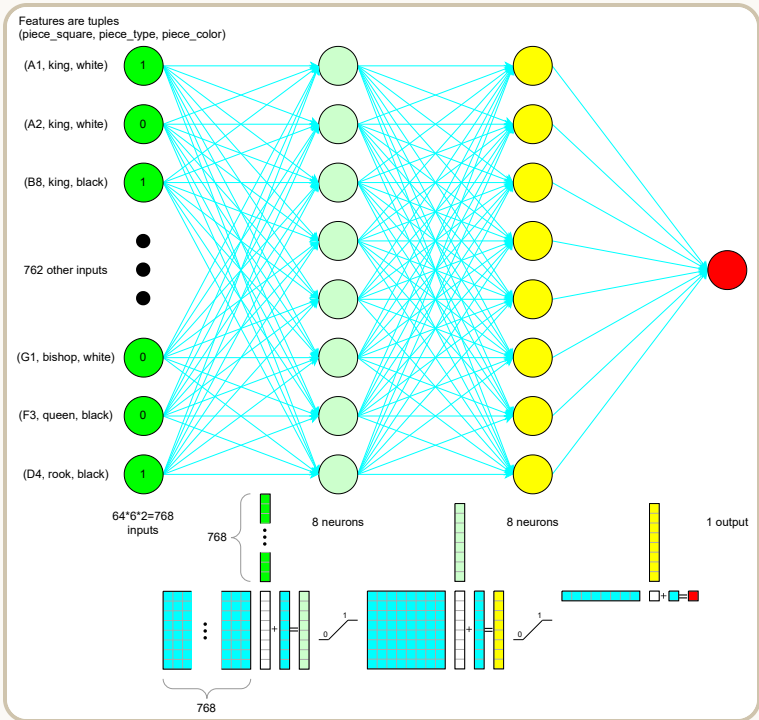
Captured piece: subtract its feature vector too.

An ordinary non-king move usually changes only these few inputs.

Moves and captures incrementally update only changed first-layer columns.

The accumulator still needs an output network

Incremental updates save first-layer summation; this project minimizes the remaining network to preserve search depth.



- Dark green: piece-square features
- Light green: incremental first layer A
- Yellow: subsequent network
- Red: final score

Official chess NNUE diagram; this project reuses the incremental wide-layer idea with a smaller direct output head.

Stockfish nnue-pytorch
documentation · GPLv3

- First layer: 11,340 sparse features share weights for two H16 accumulators.
- Activation: CReLU clips each component to its valid range.
- Direct head: concatenate mover/opponent views; select a middlegame/endgame head to predict the PST residual.

An H16 direct head learns interactions while keeping per-node inference cheap.

Addition alone is still just a larger table

Nonlinearity lets two features together trigger an effect neither creates alone.

Feature x	Feature y	x + y	h = max(0, x + y - 1)
0	0	0	0
1	0	1	0
0	1	1	0
1	1	2	1

$$h = \max(0, x + y - 1)$$

h activates only when both x and y are present.

Linear sums give each piece-square feature a fixed contribution and collapse back into a PST.

Nonlinear activation makes a feature's effect depend on other active features.

Incremental sums preserve speed; nonlinearity goes beyond fixed square bonuses.

Where does intuition come from?

Can data learn the table values?

Represent each historical position as x and its eventual result as y .

Sample x

The board and required
side-to-move
information

Label y

Red win = 1; draw =
0.5; Red loss = 0

Parameters θ

Material values, PST
entries, or weights of a
richer model

The model assigns the position a score $s_{\theta}(x)$.

Map it to Red's expected score in $[0, 1]$.

Train predictions toward historical labels.

Training fits parameters to positions with outcomes.

From evaluation to expected score

Sigmoid maps any score to 0–1

Evaluations may be hundreds or thousands; result labels are 0, 0.5, or 1.

```
predicted_score(x) = sigmoid(s(x) / K)
```

```
sigmoid(z) = 1 / (1 + exp(-z))
```

Very negative

Prediction
approaches 0

Near zero

Prediction is about
0.5

Very positive

Prediction
approaches 1

K controls how much evaluation change produces a substantial prediction change.

Estimate K on separate calibration data, then freeze it during network training.

The output is Red's predicted score.

Sigmoid aligns evaluation and outcome scales; independently calibrate and freeze K.

Reducing error

MSE penalizes prediction error

Compute mean squared error over a batch and update parameters along its gradient.

```
loss = mean((predicted_score(x) - y)2)
```

Example: prediction 0.8, Red loses ($y=0$)

Error = $(0.8 - 0)^2 = 0.64$

Batch → mean loss → gradient → update θ

Square each prediction-minus-result error, then average the batch.

Predicting 0.8 for an actual Red loss contributes 0.64.

The entire batch's gradient determines the update direction.

MSE measures prediction error; batch gradients determine parameter updates.

Teach a model to recognize what search discovers

The project connects PST teacher search, million-position datasets, NNUE learning, and quantized deployment.

Position x

↓ PST teacher search D3 / D4, selective pruning disabled

Search score from the moving side's view

↓ Learn teacher - PST residual

Quantized value model $v\theta(x)$ + original PST

Teacher

Keep alpha-beta and quiescence; disable selective pruning that could bias labels.

Data

One million unique positions each for D3 and D4; 500k per side; split by whole game.

Student

Learn the teacher's correction to PST; quantized CPU inference remains inside search.

Depth is a hyperparameter: D3 is cheaper and easier to fit; D4 is stronger but more expensive.

Deduplicate globally; resolve conflicting labels explicitly or discard the group.

Add a centipawn residual loss to probability error to avoid near-zero corrections.

Quantize to integer weights and test in the same searcher as PST under equal time.

Search distillation yields a value model cheap enough for live CPU search.

Bigger networks and deeper teachers can cost strength

Offline error screens candidates; quantized equal-CPU-time matches make the final decision.

Experiment	Observation	Decision
Old 150k one-sided D3 + H8	Little data, recording only one side to move	Expand to one million, balanced by side
Million-position D3: H8 / H16	Easier to fit, weaker in matches than the best D4 model	Use for broad pretraining
Million-position D4: random / D3 initialization	H16 with D3 initialization has the best validation correlation and match performance	Select as final candidate
Old D5-H8	Deeper but more expensive and harder for a small network; Swiss score 48.13%	Do not default to the deepest teacher
Small hidden layer	Adds cost to every leaf evaluation	Choose a direct head this round

- Effective: more balanced data, D3-to-D4 initialization, H16 direct head, residual learning.
- Not selected: small one-sided data, depth alone, extra hidden layers.
- H16's offline gain is modest; direct matches show it covers the speed cost.

Balanced data, moderate teachers, curriculum initialization, and a light head worked best.

First deployed NNUE scores 59.77% against PST

Same search, same CPU time, 192 color-swapped held-out openings; evaluation is the core variable.

172 wins / 115 draws / 97 losses

Win=1, draw=0.5:
NNUE scores 229.5 / 384 points.

59.77% score rate

9.77 percentage points above 50%; 95% CI 56.38%–63.15%.

H16 vs H8: 54.17%

149 wins, 118 draws, 117 losses; the wider model compensates for its small speed cost.

363 KB integer model

Mean completed depth 9.24 vs PST 9.70: stronger evaluation costs about 0.46 ply.

192 openings, each played with both colors, produce 384 games.

Bootstrap opening pairs to retain the correlation between swapped games.

H16 scores 54.17% against H8 with only 0.06 ply difference in mean depth.

After quantization, first-generation NNUE lifts the equal-time score against PST to 59.77%.

Search D3 with NNUE, then teach the results back

A deeper PST teacher is costly; one best-NNUE + D3 iteration generates a new million-position dataset more cheaply.

Configurable teacher

One generator selects PST or a supplied NNUE; PST remains the compatible default.

Balanced million

One million unique positions, 500k per side; only 12 conflict groups discarded.

Vs previous model: 60.94%

174 wins, 120 draws, 90 losses; 95% CI 57.16%–64.71%.

Vs PST: 63.28%

188 wins, 110 draws, 86 losses; up 3.52 points from 59.77%.

The quantized D4-H16 evaluates leaves in a three-ply teacher search.

Initialize from the current model, independently calibrate K; best validation at epoch 4.

The student scores 60.94% against its predecessor and 63.28% against PST.

The first NNUE+D3 iteration beats its predecessor and raises the PST score to 63.28%.

Repeat the cycle: 70.18% against PST

Iter1 supplies both teacher and initialization; generate a fresh million positions with unchanged architecture and match conditions.

A new million

Sample exactly one million from 1,057,865 unique valid positions; 500k per side; 15 conflict groups discarded.

Independent K=54.708

Calibrate on this iteration's data; best epoch 5, early stop at 25, restore best checkpoint.

Vs Iter1: 55.08%

143 wins, 137 draws, 104 losses; 95% CI 51.43%–58.72%, entirely above 50%.

Vs PST: 70.18%

218 wins, 103 draws, 63 losses; 95% CI 66.80%–73.44%, up 6.90 points.

Iter1 evaluates leaves while D3 search generates new labels.

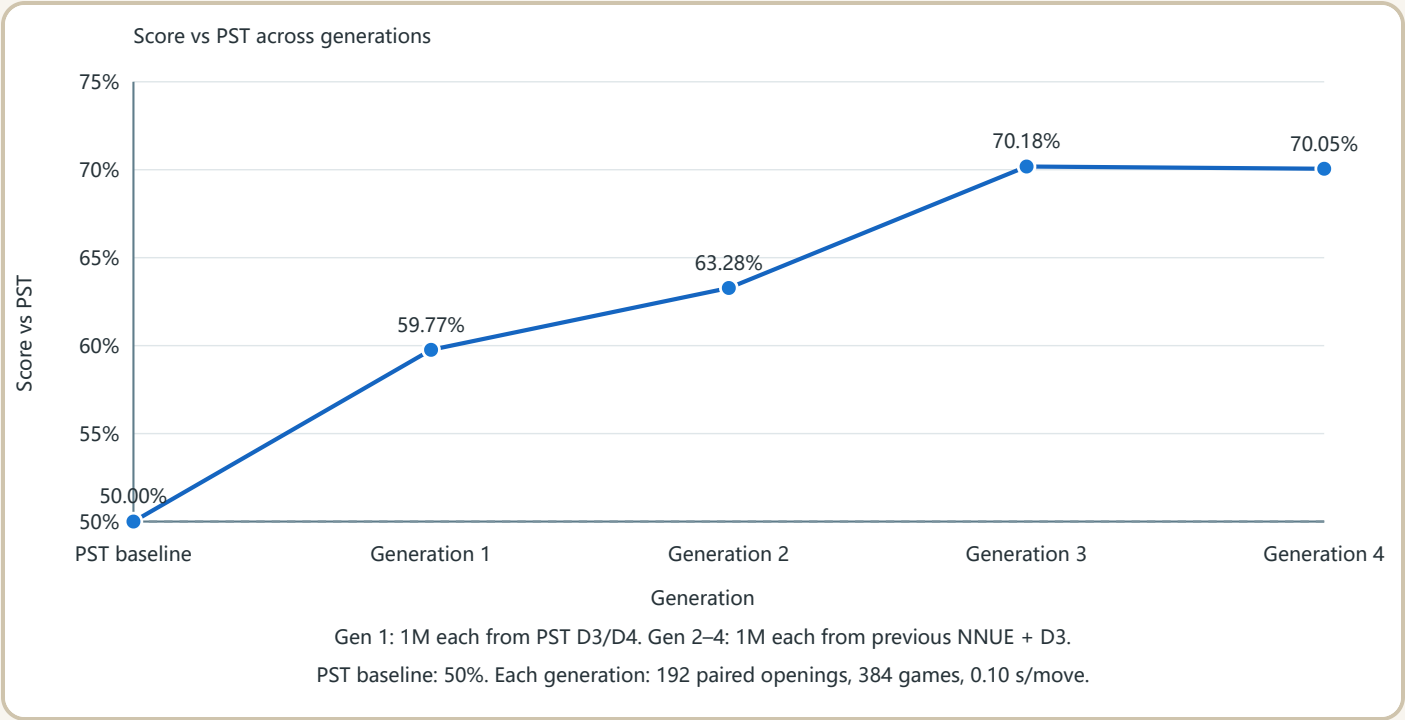
Iter2 starts from Iter1 parameters; best validation at epoch 5.

Iter2 scores 55.08% against Iter1 and 70.18% against PST.

The second iteration beats Iter1 and reaches 70.18% against PST under the same conditions.

Four generations reach 70.18% against PST

With architecture, held-out openings, and time controls fixed, generation 3 has the highest observed score.



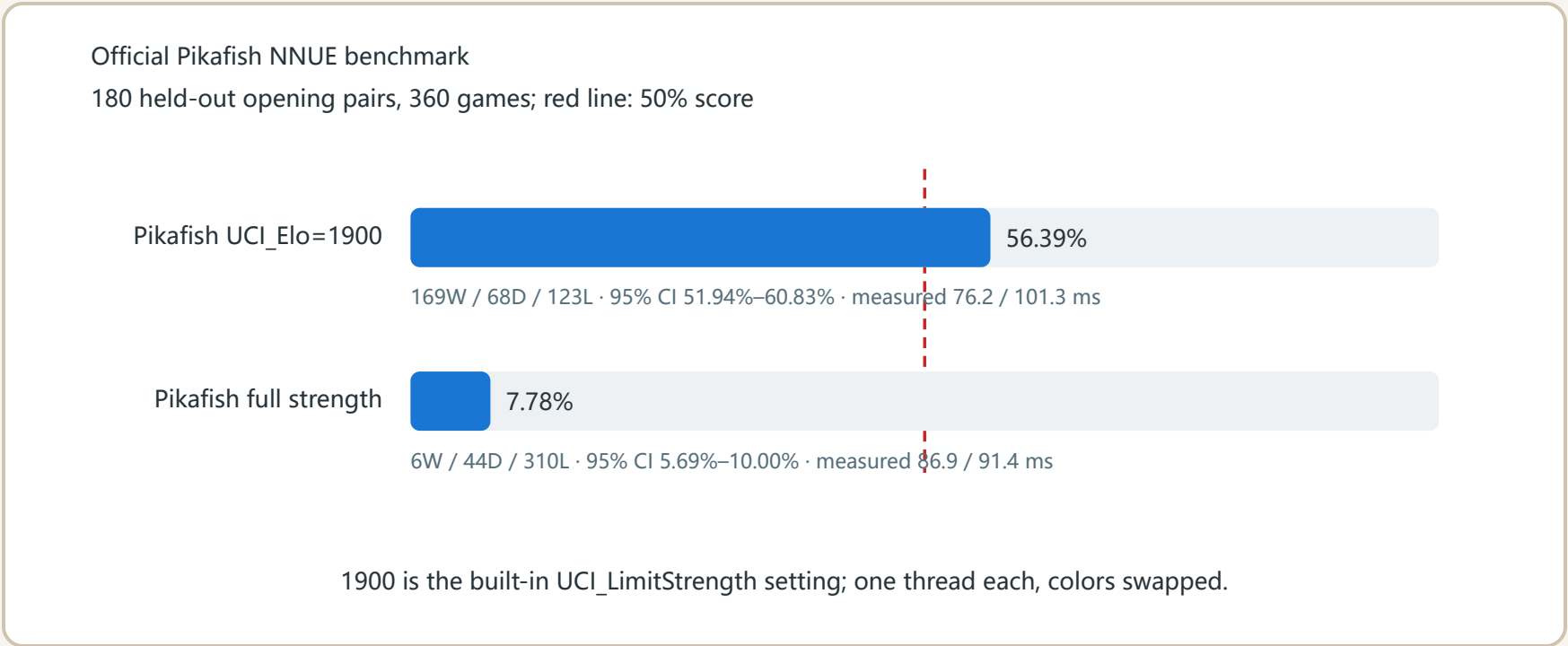
192 color-swapped held-out openings; 384 games per generation, 0.10 s per move, one CPU core.

Generated from experiment JSON

Teacher iteration improves the small H16 model; generation 3 reaches 70.18% under one protocol.

Above Pikafish's built-in 1900 setting

On 360 held-out color-swapped games, our NNUE scores 56.39%; full-strength Pikafish shows the remaining gap.



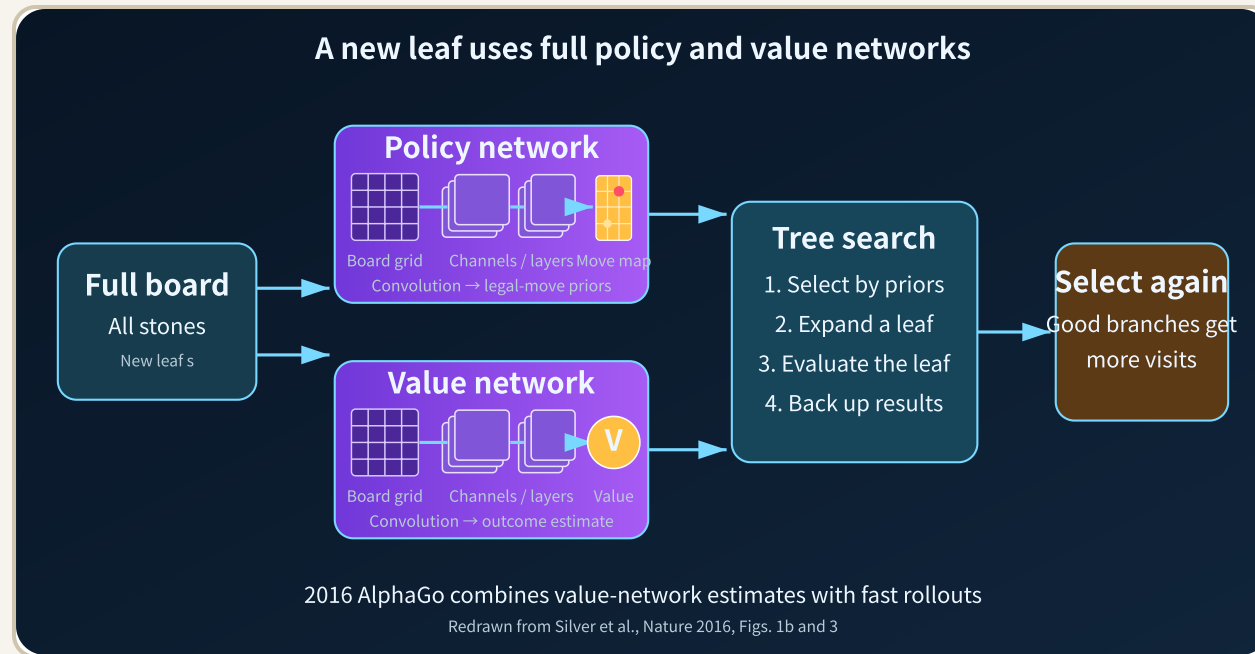
Official Pikafish 2026-01-31; 180 formal openings, each color-swapped, 360 games.

Per-game JSON and paired bootstrap

Our NNUE exceeds Pikafish's built-in 1900 setting, with a large gap to full strength.

Policy guides, value evaluates, the tree backs up results

CNNs scan local board windows and combine features across layers; each new leaf's output guides subsequent tree visits.



Policy: prioritize legal moves and guide the tree.

Value: estimate a new leaf's outcome prospects.

Back up evaluations so stronger branches receive more visits.

Redrawn from 2016 AlphaGo, Figs. 1b and 3; that version combines value predictions with fast rollouts. Silver et al. · Nature 2016

Policy and value guide the tree with richer, more expensive full-board inference at new leaves.

Why is NNUE cheaper inside this CPU searcher?

Incremental evaluation leaves more CPU time for exploring continuations.

Approach	Reuse between adjacent positions	Cost of a new node	Effect under fixed time
NNUE + traditional search	Subtract old features, add new ones; reuse the first-layer accumulator	Low CPU latency, followed by a small output network	Evaluate many nodes and search deeper
AlphaGo-style CNN + MCTS	Run full-board convolution at each new leaf; cache results in the tree	Heavier leaf inference; the original system used asynchronous GPUs	Fewer new leaves; policy priors focus visits on promising branches

Pikafish Wiki · 2025-01-06

The wiki reports that CPU engines remain stronger than GPU engines in Xiangqi.

NNUE caches the expensive first layer, updates changed features, and runs small later layers.

An AlphaGo-style CNN performs multilayer full-board inference at each new leaf.

On the same CPU budget, heavier nodes mean fewer replies explored and less reachable depth.

NNUE spends the budget on more nodes; the AlphaGo approach uses richer guidance per leaf.

What do we use after these experiments?

Keep PST as a transparent baseline; deploy the trained and validated NNUE as the default upgrade.

Option	Project result	Current use
Material only	Piece types and counts; incremental O(1) updates	Simplest evaluation baseline
PST	Fast, interpretable ElephantEye tables adjusted through practice	Keep as the project PST engine and comparison baseline
Piece relationships	The attempted features made execution several times slower	Removed
Project NNUE	D3/D4 training and three NNUE+D3 teacher iterations completed	Iter2 peaks at 70.18%; deployed in the web demo

Explicit relationship terms added information but cost several times more runtime.

PST remains interpretable and makes neural gains measurable.

Iter2 has the highest observed score across three teacher iterations; deploy.ps1 updates the demo.

PST is the transparent baseline; NNUE is the validated default. Evaluate both with search cost.

Sources: xiangqi_ai.cpp:791

Stage 2 complete

We have intuition, but not foresight

Neither PST nor search-distilled NNUE replaces explicit consideration of replies.

What we have

Two evaluators: fast, transparent PST and incremental NNUE that learns relationships.

What is missing

How will the opponent reply? Will a captured piece immediately be recaptured?

Stage 3

Expand moves into a tree and test intuition against both players' choices.

Evaluation measures positions reached by search.

Search determines which positions can actually arise.

Evaluation scores leaves; search tests those scores against concrete replies.

Sources: xiangqi_ai.cpp:791 · xiangqi_ai.cpp:1123

We can score positions, but cannot yet play

We have a board, rules, and evaluation. Now include the opponent's replies.

What we can do

Generate legal moves,
make/undo, and
evaluate leaves

What is missing

The opponent
deliberately chooses
replies that hurt us

This chapter

Understand counterplay
within a limited time
budget

First, enumerate our candidates.

Then add a layer: the opponent chooses too.

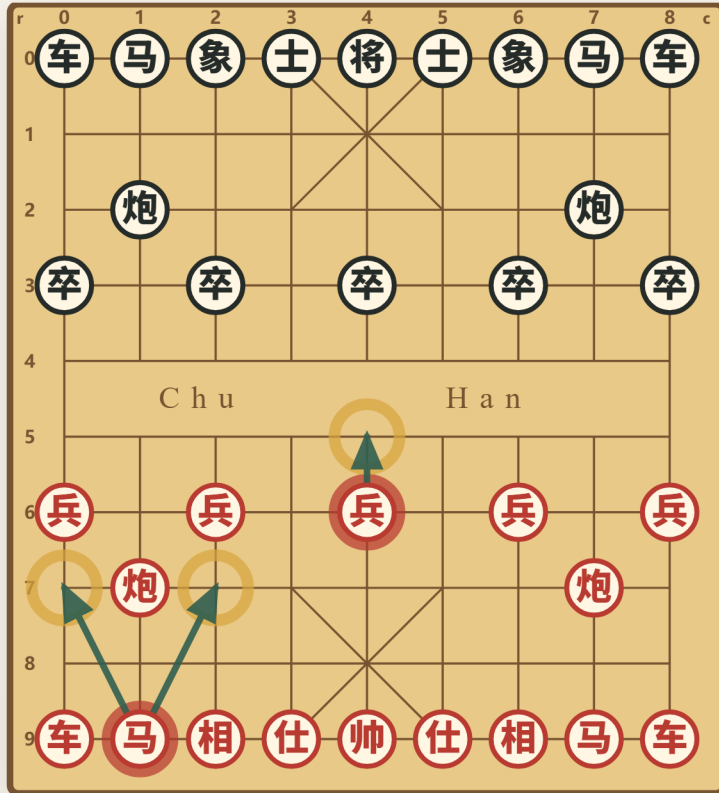
Finally, face tree growth and deadlines.

Search places evaluation inside alternating choices by both players.

Sources: xiangqi_ai.cpp:1123

The simplest approach: try every move

Generate → make → evaluate → undo → try the next move.



Initial position: example Red candidates

```
best = -INF
for move in legal_moves:
    captured = make(move)
    score = evaluate()
    undo(move, captured)
    best = max(best, score)
```

Each candidate is a testable hypothesis.

Evaluate after make.

Return to the same position after undo.

Search begins by turning moves into reversible hypotheses.

The first failure

One-ply evaluation is greedy

Winning material now does not mean the opponent cannot take it back.

The old logic

Choose whichever move leaves the highest immediate score.

The reality

The opponent moves next and chooses our least welcome reply.

The missing skill

Enumerate the opponent's replies after our own move.

We prefer a candidate with a higher immediate score.

An opponent reply may reverse that advantage.

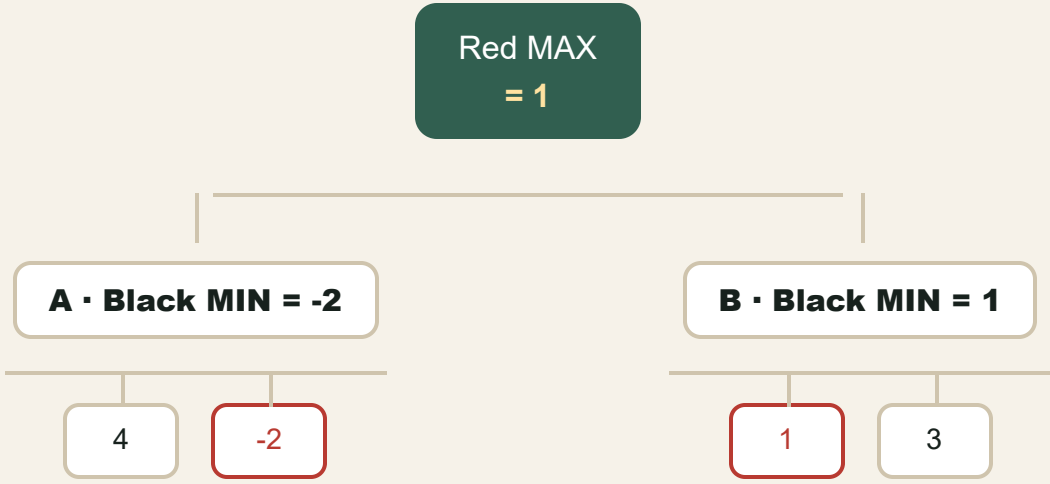
At the next layer, the opponent chooses what is worst for us.

The opponent chooses the reply most unfavorable to us.

Sources: xiangqi_ai.cpp:1123

One score perspective: Red maximizes, Black minimizes

Scores always favor Red when positive: Red takes the maximum, Black the minimum.



4 / -2 / 1 / 3 are teaching values, not real game scores

- All four leaf values use Red's perspective.
- Black takes $\min(4, -2) = -2$ and $\min(1, 3) = 1$.
- Red takes $\max(-2, 1) = 1$ at the root.

Minimax encodes optimal opponent choice in recursion.

Sources: xiangqi_ai.cpp:1123 · xiangqi_ai.cpp:1409

The recursive structure

Repeat the same logic at every layer

Make, switch sides, search, undo; maximize and minimize alternate.

```
search(position, depth, red_turn):  
    if terminal: return win_or_loss_score(position)  
    if depth == 0: return evaluate(position)  
  
    best = red_turn ? -INF : +INF  
    for move in legal_moves:  
        captured = make(move)  
        score = search(position, depth - 1, !red_turn)  
        undo(move, captured)  
        best = red_turn ? max(best, score) : min(best, score)  
    return best
```

Stop at a terminal outcome or the depth limit.

Make before recursion and undo afterward; switch the side to move.

Red takes max; Black takes min.

A game tree repeats make → switch sides → undo.

Sources: xiangqi_ai.cpp:1123 · xiangqi_ai.cpp:1371 · xiangqi_ai.cpp:1446

Give checked positions a little more room

Extend one ply if the boundary cuts off a forced reply, with a bounded extension budget.

Why extend?

A checked side has forced replies; static evaluation may stop mid-sequence.

How to limit it

Start with a depth-dependent budget and consume it on each extension.

Scope

Limited extension reduces forced-line truncation but remains budgeted.

Depth reaches zero while the side is in check.

With remaining budget, increase effective depth by one.

Pass the reduced extension budget into recursion.

Extensions prioritize forcing lines while controlling total cost.

Sources: xiangqi_ai.cpp:1127

What if depth eight is unfinished at the deadline?

Time may expire after only some root candidates have been searched.

Fixed target

Starting directly at depth 8 may finish only the first few root moves.

Interrupted state

Unsearched root moves prevent a complete comparison at this depth.

A better target

Keep an answer from the latest fully completed depth at all times.

Depth 8 may time out before all root candidates are compared.

Stop this iteration and retain the previous complete result.

Complete depths in order so every interruption has an answer.

Time management must decide what to return when the search is interrupted.

Meeting the deadline

Iterative deepening: save each completed answer

Search depths 1, 2, 3...; on timeout return the latest completed iteration.

Depth	Status	Best move	Score	Action
1	Complete	M1	S1	Save
2	Complete	M2	S2	Replace the previous complete answer
3	Complete	M3	S3	Replace the previous complete answer
4	Timeout	Incomplete	Incomplete	Discard this iteration; return M3

- Save the full result after depths 1, 2, and 3.
- Depth 4 times out midway and is not the final answer.
- Return the last complete result from depth 3.
- The previous best move improves ordering at the next depth.

Iterative deepening provides interruptibility and move-ordering experience.

Check the deadline every batch of nodes

Check the hard deadline every 2048 nodes; after each iteration, decide whether to start another.

```
if ((nodes & 2047) == 0):  
    if elapsed > time_limit:  
        stop_search = true  
  
for depth = 1..63:  
    result = search(depth)  
    if stop_search: break  
    last_res = result  
    if elapsed > max_time * 0.16 && depth >= 4:  
        break
```

Read the clock at the node-check interval.

Propagate the timeout signal up the recursion.

Update last_res only after a complete iteration.

If one iteration uses over 16% of the total budget, do not start the larger next one.

Check time throughout search and retain the last complete result.

Recursion makes the tree explode

With b candidates and depth d , naive work grows roughly as b^d .

One more ply

Every existing leaf branches again.

Leaves may be unstable

Later, captures may extend the search beyond the normal boundary.

Teaching example

At 30 moves per position, depth 4 has about 810,000 leaves; depth 6 has 729 million.

The real question

Do we need the exact score of every candidate?

Each additional layer branches at every leaf.

For $b=30$: $b^4=810000$ and $b^6=729000000$.

Ask whether choosing the best move requires completing every branch.

Exponential branching dominates the cost of individual evaluations.

How do we accelerate the search?

Use bounds, stable leaves, position memory, and ordering to reach conclusions sooner.

Method	Purpose
Alpha-Beta	Stop remaining branches when bounds already determine the choice
QS	Continue tactical moves at leaves; generate all evasions when checked
SEE for QS filtering	Filter clearly losing captures when not in check
Zobrist key	Identify positions with a compact, incrementally updated fingerprint
TT lookup	Use the key to locate previous search records
TT reuse	Reuse sufficiently deep exact scores or bounds
TT replacement	Retain valuable records within fixed capacity
TT move	Try the cached best move first
Capture ordering + SEE	Try more promising exchanges first
Killer	Prioritize quiet moves that previously cut off at this ply
Counter	Prioritize moves that successfully answered the previous move
History	Order quiet moves by accumulated success

- Reduce branches needed to reach the same conclusion.
- Reuse searched positions and try promising moves earlier.
- SEE serves both QS filtering and capture ordering.

Avoid redundant expansion and establish useful bounds earlier.

Next: probe, reduce, recover, and look up

Cheap probes and selective search focus the budget; recovery and tables control costs.

Method	Purpose
IID	Without a guide move, search shallowly to improve ordering
PVS	Full window for the first move; narrow probes for others
Aspiration	Start with a narrow window around the previous iteration's score
LMR	Reduce depth for late, unpromising moves
LMP	Skip very late quiet moves at shallow depth
Futility	Skip shallow quiet moves whose optimistic estimate still falls short
SEE pruning	Skip clearly losing exchanges at shallow depth
RFP	Cut off when the static score clears the bound by a margin
Razoring	Verify poor shallow positions with QS
NMP	Probe a reduced-depth null move for an early cutoff
Second-pass recovery	When triggered, rerun with three local skipping rules disabled
Rook/cannon attack tables	Maintain occupancy incrementally and look up precomputed attacks

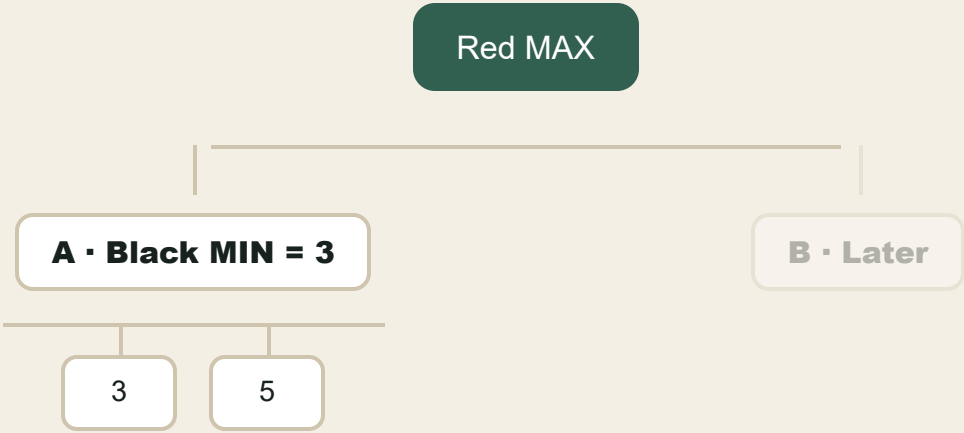
- Cheap probes ask whether full search is worthwhile.
- Selective methods focus the budget on likely critical branches.
- Recovery reduces missed-move risk; attack tables reduce repeated query cost.

Selective search focuses effort; recovery and tables control risk and per-query cost.

First cutoff · Establish a reference

First solve A: what will the opponent allow?

Red considers A; Black chooses the smaller of 3 and 5.



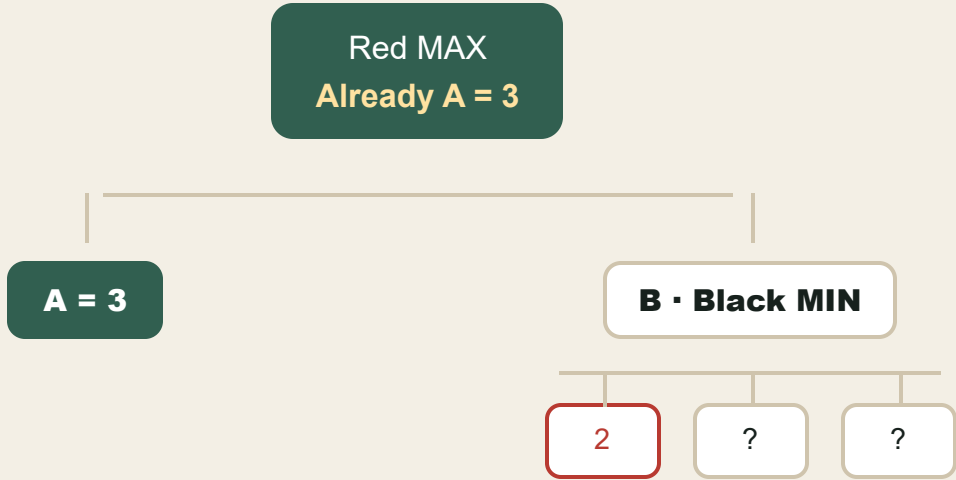
3 / 5 / 2 are teaching values, not real game evaluations

- Black's first reply under A scores 3.
- The second scores 5.
- Black takes $\min(3,5)=3$, giving Red an option worth 3.

Solve one path to establish a comparison bound.

B is unfinished. Do we already know enough?

B's first reply scores 2; a later reply might score 10. Continue?



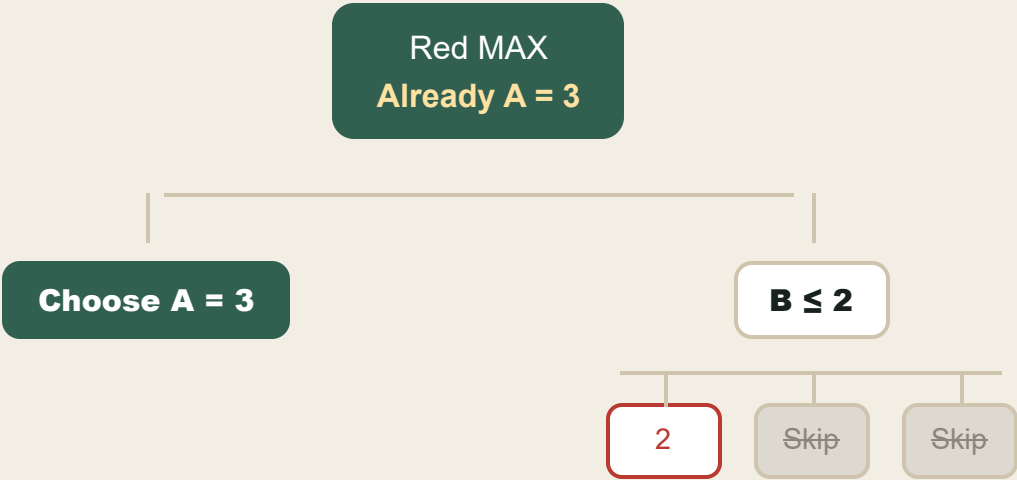
A=3 is known; B's MIN node has already seen 2

- The first opponent reply under B scores 2.
- Later replies remain unknown.
- Who chooses among B's replies?

A bound can justify stopping before all exact values are known.

We do not know B exactly, but will not choose it

Red already has A=3; Black can hold Red to at most 2 under B.



Skip the unknown branches: B's upper bound cannot challenge A

- The remaining B branches cannot change the root choice.
- B's exact score is still unknown.
- Alpha-beta uses bounds to decide when to stop.

When bounds determine the choice, exact values are unnecessary.

Alpha and beta are decision thresholds

A MAX ancestor has an alternative worth at least alpha; a MIN ancestor has one worth at most beta.

Alpha: lower threshold

A MAX ancestor can get at least alpha elsewhere, so it rejects anything lower.

Beta: upper threshold

A MIN ancestor can hold Red to at most beta elsewhere, so it rejects anything higher.

The useful interval

Only a value between alpha and beta can change the ancestors' choices.

Alpha comes from MAX's existing alternatives.

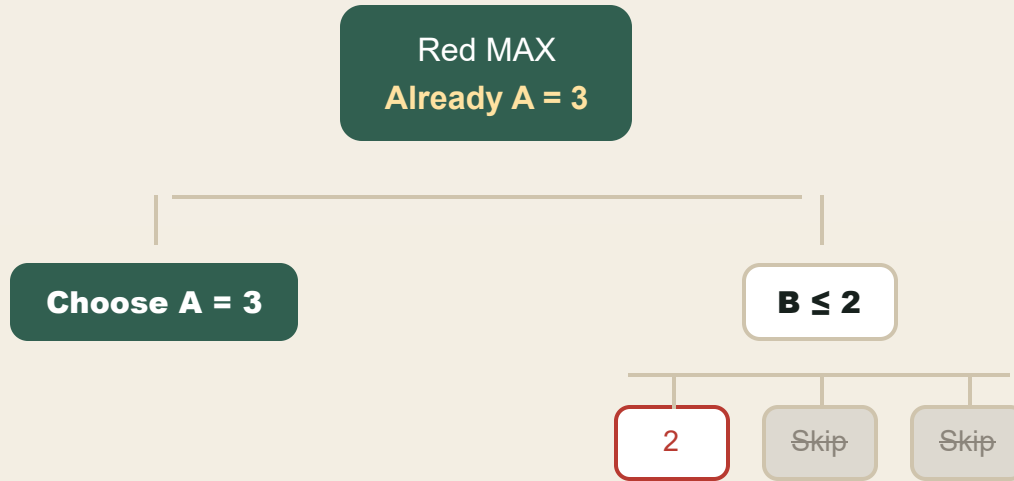
Beta comes from MIN's existing alternatives.

Recursion passes both as the current search window.

Alpha and beta record thresholds set by ancestor alternatives.

Translate A and B into alpha and beta

A=3 sets alpha=3 at the root; B's first reply 2 lowers beta to 2.



A=min(3,5)=3; B's MIN node first sees 2

```
alpha = 3           // Root MAX already has A=3
beta = +INF          // Enter B's MIN node

score = 2            // First Black reply under B
beta = min(beta, score) // beta becomes 2

if alpha >= beta:     // 3 >= 2
    break             // Stop remaining B branches
```

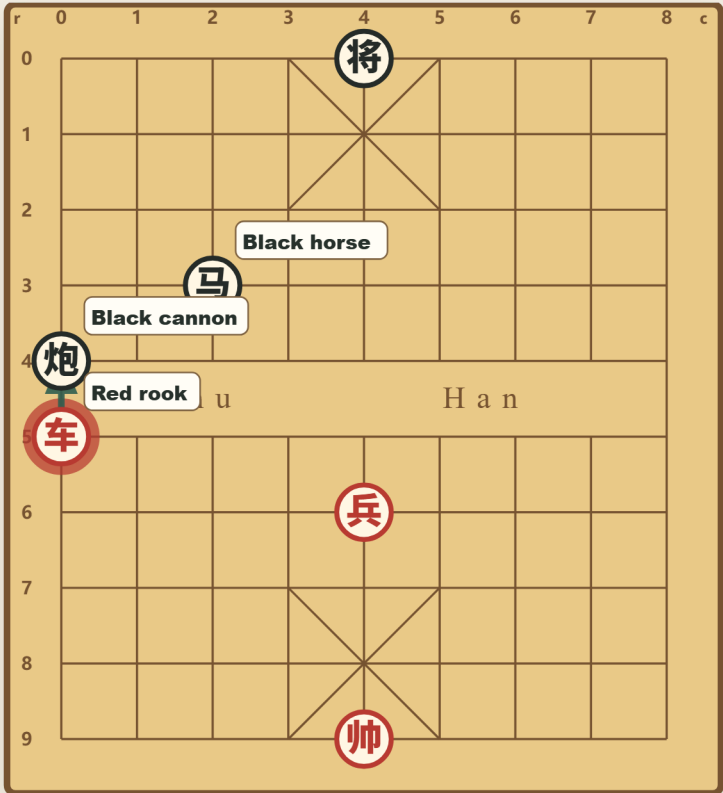
A=min(3,5)=3 raises root alpha to 3.

B's first reply makes beta=2; alpha≥beta, so skip both unknown branches.

When the bounds cross, this branch cannot affect the ancestor choice.

Depth runs out before the exchange ends

A fixed depth counts plies; it does not wait for captures and recaptures to finish.



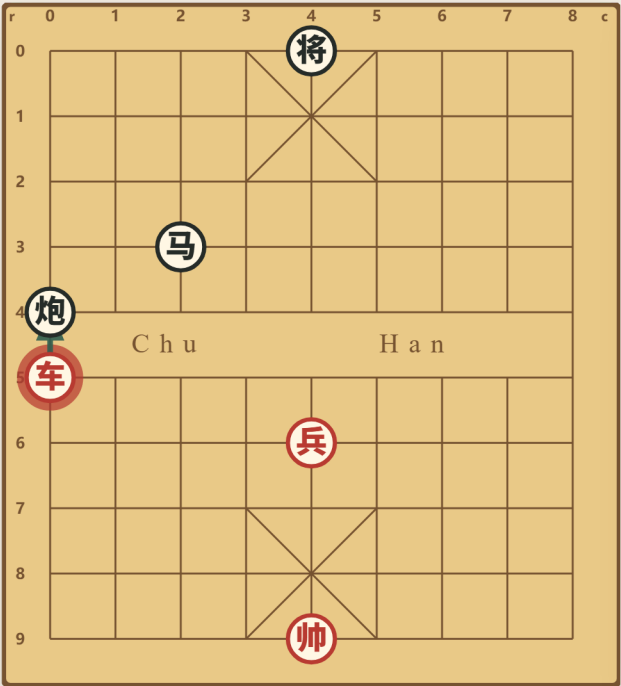
Before: Red to move; the rook can capture the black cannon

- Red's rook advances one square and captures the cannon.
- This move consumes the remaining normal depth.
- Black's horse can still recapture through the empty leg square (3,1).

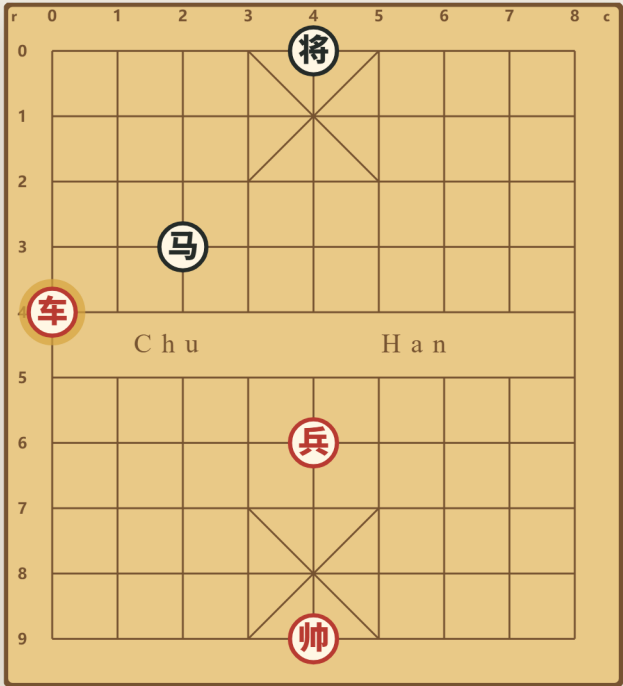
A depth boundary is artificial; tactical exchanges need not end there.

Frame 1: the rook takes the cannon

Static evaluation records the captured cannon before the recapture enters the leaf score.



Before



After: (5,0) empty, red rook at (4,0), black cannon removed

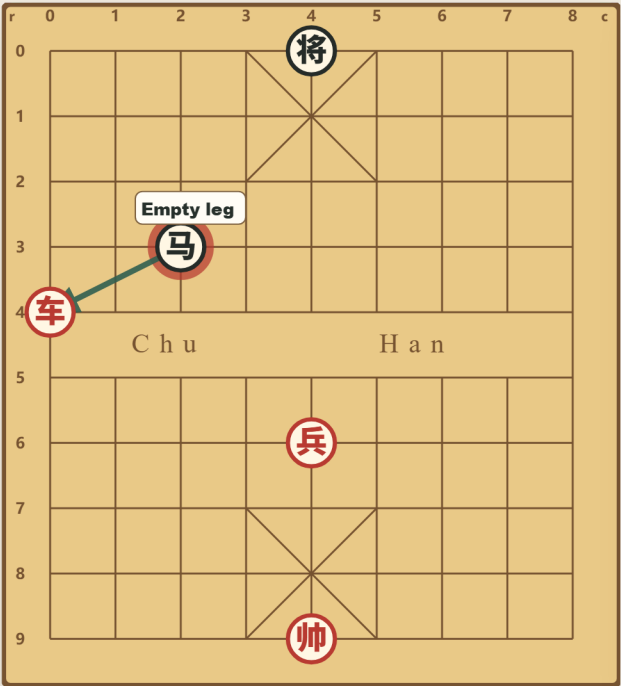
- The leaf temporarily contains one fewer black cannon.
- Static evaluation immediately reflects the material gain.
- Black's possible recapture determines whether that gain survives.

A high post-capture score may be an unfinished-exchange illusion.

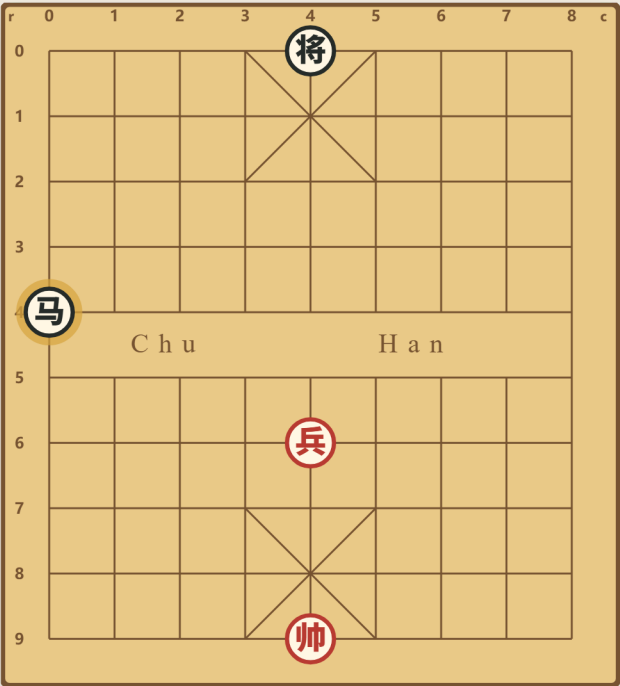
Sources: xiangqi_ai.cpp:434

Frame 2: the horse recaptures the rook

The horse jumps through an empty leg square, removes the rook, and occupies its square.



Before recapture: Black to move



After: (3,2) empty, black horse at (4,0), red rook removed

- The empty leg allows the horse to recapture.
- The horse occupies the target and the rook disappears.
- Base material: Red gains cannon 450, loses rook 1000, net -550.

Follow captures and recaptures to a relatively stable position before evaluating.

Sources: xiangqi_ai.cpp:669 · xiangqi_ai.cpp:1030

Why not deepen Minimax everywhere?

Full-width deepening expands every legal move at every leaf; only unfinished tactical exchanges need attention here.

One extra full layer

All legal rook, horse, cannon, and pawn moves branch, including quiet ones.

Two extra full layers

Every new node expands all candidates again, multiplying work.

The exchange may still continue

A fixed extension merely moves the horizon; a longer exchange can cross it too.

Selective continuation

After normal depth ends, continue captures and recaptures.

Full-width deepening expands quiet and tactical moves alike.

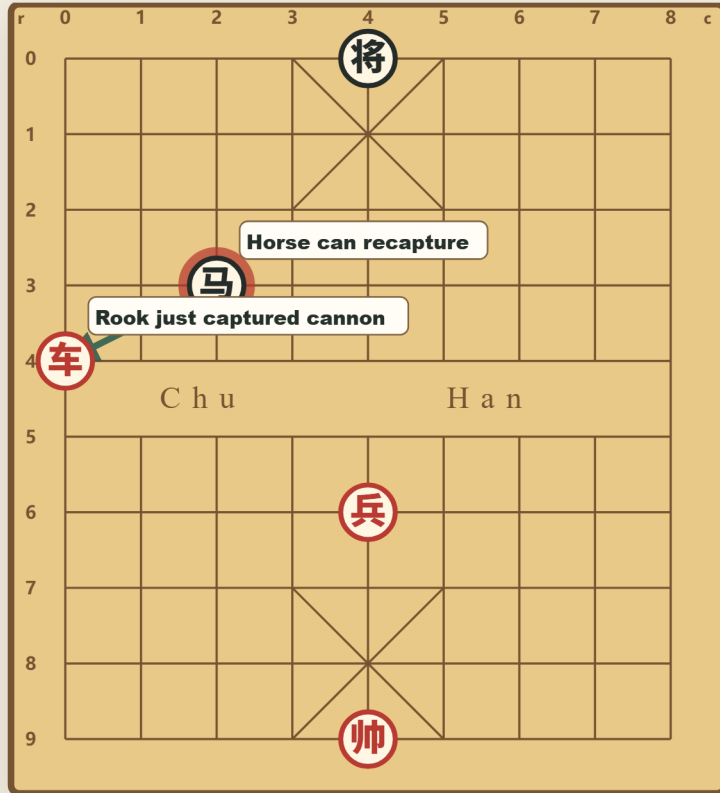
A fixed extra depth can still stop mid-exchange.

Concentrate extra work on unfinished exchanges, usually with fewer branches.

Spend additional depth first on unfinished forcing lines.

At depth zero, QS takes over

After the rook takes the cannon at normal depth zero, QS continues with Black to move.



Normal depth=0 · Black to move

```
if (depth <= 0) {  
    return qs(alpha, beta, maximizingPlayer, 0);  
}
```

Normal search stops just after the rook capture.

QS begins at qsDepth=0 and explores the horse recapture.

QS is another recursive search entered at normal-search leaves.

At each QS entry, choose candidates and stopping rules

Without check, tighten the window using stand pat; in check, find evasions; qsDepth bounds extra work.

Current state	QS action
Not in check	Evaluate stand pat; if no cutoff, prepare captures
In check, qsDepth 0–3	Skip stand pat; prepare all moves, then filter legal evasions
In check, qsDepth 4–6	Check whether any legal evasion exists; evaluate if yes, return loss if none
qsDepth > 6	Return evaluate() and stop QS

```
qs(alpha, beta, maximizing, qsDepth):
    inCheck = is_in_check(maximizing)
    if !inCheck:
        standPat = evaluate()
        if maximizing:
            if standPat >= beta: return beta
            alpha = max(alpha, standPat)
        else:
            if standPat <= alpha: return alpha
            beta = min(beta, standPat)

    if qsDepth > 6: return evaluate()
    if inCheck and qsDepth > 3:
        if any legal evasion: return evaluate()
        return maximizing ? -30000 + qsDepth
                           : 30000 - qsDepth

    moves = inCheck ? all_moves : captures
```

Entry logic establishes the baseline, candidate set, and depth boundary before recursion.

For each candidate, make a move and call QS again

Make, reject illegal moves, recurse, undo, and update the node's window.

```
hasLegal = false
for m in moves:
    captured = make(m)
    if is_in_check(maximizing):
        undo(m, captured); continue
    hasLegal = true

    score = qs(alpha, beta, !maximizing, qsDepth + 1)
    undo(m, captured)

    if maximizing:
        if score >= beta: return beta
        alpha = max(alpha, score)
    else:
        if score <= alpha: return alpha
        beta = min(beta, score)

if inCheck and !hasLegal:
    return maximizing ? -30000 + qsDepth : 30000 - qsDepth
return maximizing ? alpha : beta
```

For a legal move, call qs with the other side and qsDepth+1.

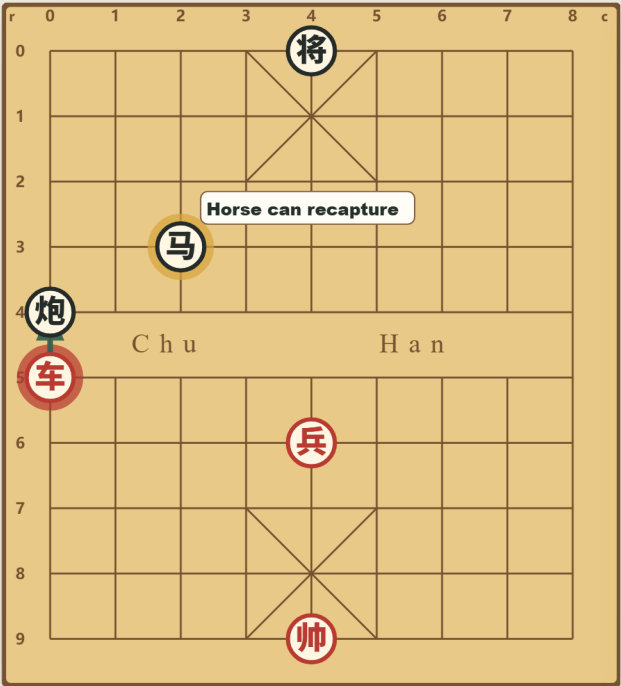
After the child returns, undo and update alpha or beta.

In check with no legal evasion, return the moving side's loss score.

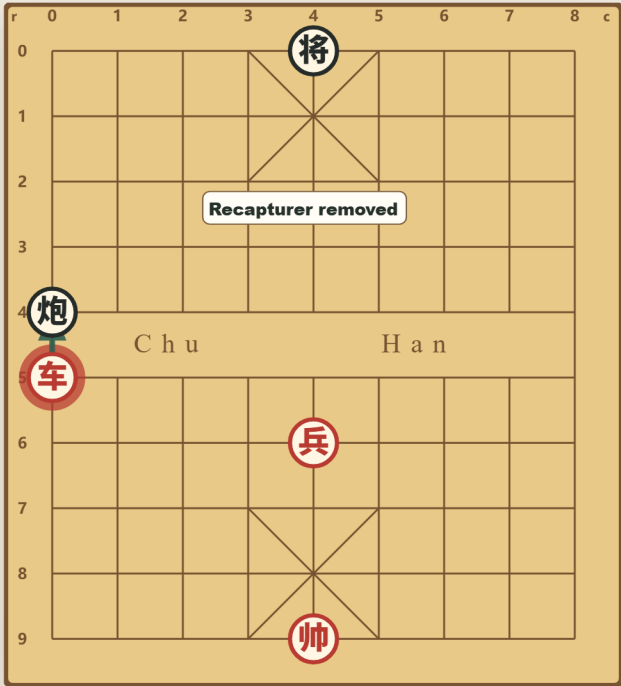
QS recursion follows exchanges; hasLegal handles check with no escape.

SEE: cheaper and coarser than QS

Estimate exchanges on one target square for QS filtering, main-search ordering, and shallow pruning.



Rook 900 takes cannon 400; horse recaptures: SEE = -500



Rook 900 takes cannon 400; no recapture: SEE = +400

SEE(move):

- Fix the initial capture's target square
- Alternate least-valuable attackers
- Back up net gains from the end

Do not expand replies away from the target square

SEE estimates alternating captures on one target instead of a full game tree.

It is cheaper than QS but misses tactics elsewhere on the board.

Both positions start with the same rook taking the same cannon.

The horse recapture alone changes +400 to -500.

SEE is a reusable local exchange estimate, not a complete search.

Turn a target-square exchange into numbers

The separate SEE table uses rook 900, horse/cannon 400; it excludes PST and differs from earlier teaching values.

```
gain[0] = see_value(victim);           // Initial captured piece
on_sq = see_value(attacker);           // Piece now on target
side = opponent;

while ((an = attackers_to(target, side, occupancy)) > 0) {
    best = least_valuable_attacker;
    gain[n] = on_sq - gain[n - 1];
    on_sq = see_value(best.piece);
    remove best's source from occupancy; // Reveal sliders
    side = !side;
}
```

Stage	Least-valuable attacker	Gain calculation	Result
Red rook takes cannon	Red rook 900	gain[0] = cannon value	400
Black horse takes rook	Black horse 400	gain[1] = rook 900 – gain[0] 400	500
No further attacker	Exchange chain ends	Begin backward propagation	Pending

- victim is the initially captured piece; gain[0] stores its SEE value.
- on_sq stores the value currently occupying the target.
- attackers_to finds the side's attackers; choose the least valuable.
- Removing each attacker updates occupancy and reveals rook/cannon lines.
- This example produces raw gain = [400, 500].

The forward loop builds the exchange ledger and reveals new attacks as occupancy changes.

Either side can stop: back up from the end

A side stops an unfavorable continuation; the return value is the initiator's net gain or loss.

```
// Forward result: gain = [400, 500]
gain[0] = -max(-gain[0], gain[1]);
         = -max(-400, 500);
         = -500;

return gain[0]; // Net SEE of rook taking cannon
```

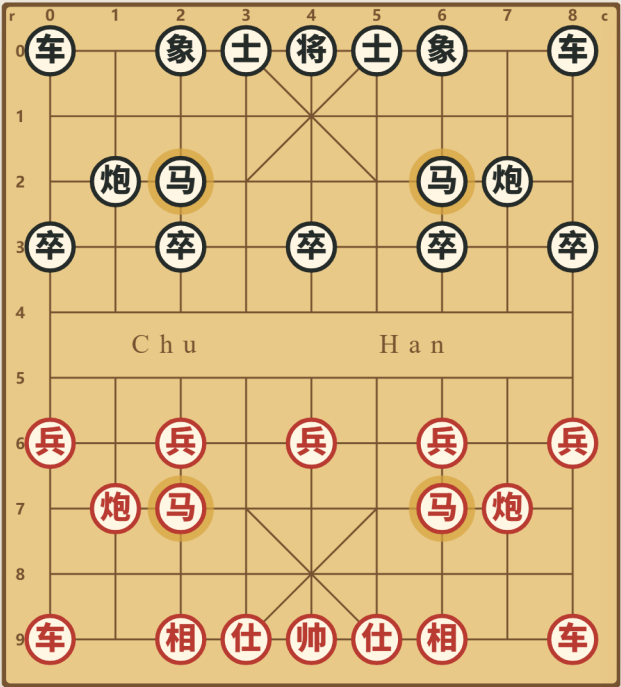
Use	Condition	Action for SEE = -500	Purpose
QS leaf filtering	Not in check; valuable attacker takes cheaper victim; threshold 0	Exclude from QS candidates	Filtering: no QS recursion
Main-search capture ordering	Valuable attacker takes cheaper victim; threshold 0	Place in a lower ordering tier	Ordering: still searched
Shallow main-search pruning	Non-root, not in check, depth ≤4; threshold -50	Skip on first pass	Pruning: recover on specific second-pass conditions

- gain[1]=500 means Black benefits from the recapture and continues.
- Backward propagation changes the initial move's result to -500.
- SEE uses the initial capturer's perspective; negative means a net loss.
- One SEE value drives three different strengths of action.

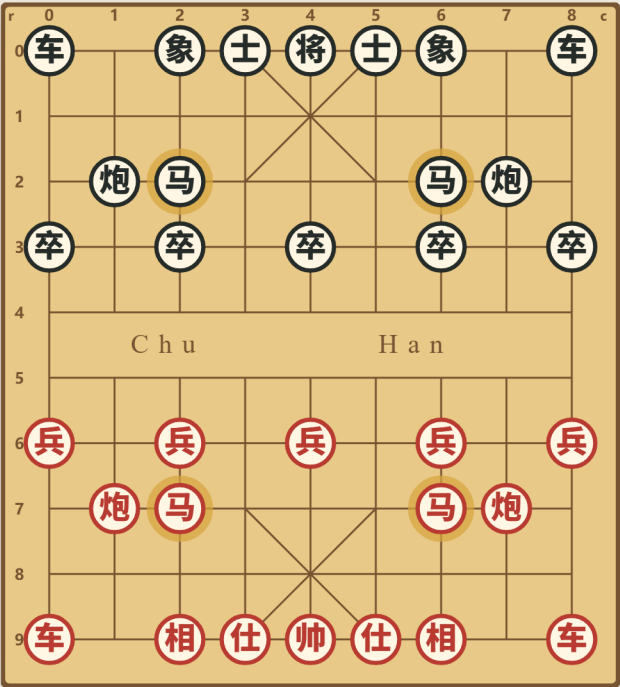
SEE returns the initiator's target-square exchange gain for filtering, ordering, and pruning.

Two move orders reach the same position

Both paths start from the initial position; reversing horse-development order reaches the same board and turn after four plies.



Path A endpoint · Red to move



Path B endpoint · Red to move

Path A

Red left horse →
Black left horse →
Red right horse →
Black right horse

Path B

Red right horse →
Black right horse →
Red left horse →
Black left horse

Convergence

The same four horse squares and Red to move

Path A develops the horses on the diagram's left first.

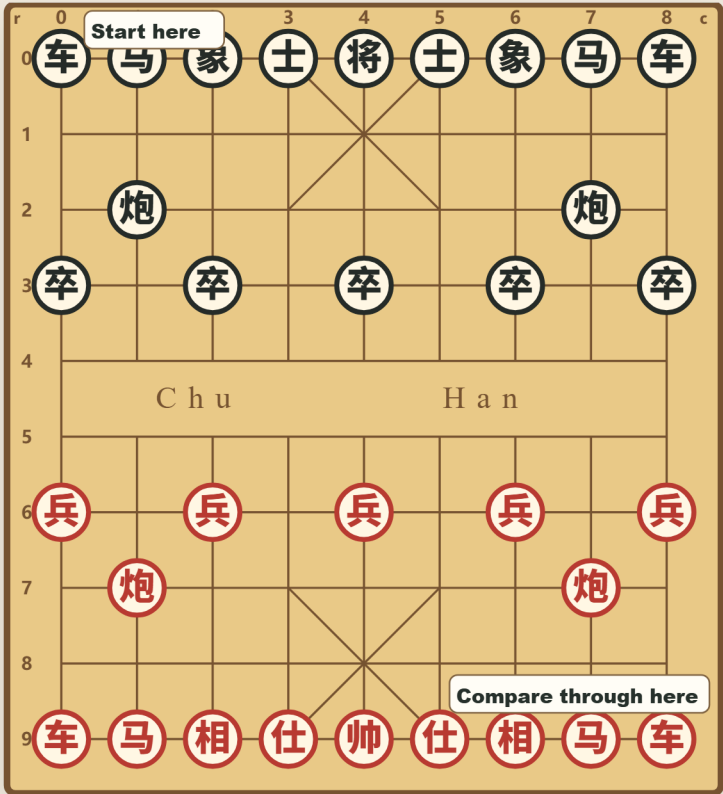
Path B develops the horses on the right first.

Identical endpoint states have identical continuation trees.

Caching avoids a duplicate subtree when move orders converge.

Can we avoid scanning 90 squares at every node?

Square-by-square comparison recognizes positions, but repeatedly scanning the whole board adds cost.



Current position · Red to move

State identity
Pieces on 90 intersections plus side to move

Naive comparison
Inspect up to 90 squares each time

What we want
Update only what actually changes during a move

The naive approach records and compares every square.

A move usually changes only a few events.

Maintain a compact fingerprint of the complete position.

A cache key must represent the full state and update cheaply.

Sources: xiangqi_ai.cpp:444

Give every piece-square event a random fingerprint

Assign a random 64-bit number to each piece-square combination, then combine active events.

Piece × Square

`Z[red rook][(9,0)]`, `Z[red horse][(9,1)]`, etc. are independent.

Board fingerprint

`H`, called `current_hash` in code, XORs all active events.

Side to move

XOR a `TURN` fingerprint when Black is to move.

Each piece-square combination gets an independent random number.

XOR all current piece events together.

Include `TURN` for Black to move.

A Zobrist key combines piece placement and side to move.

Sources: `xiangqi_ai.cpp:201` · `xiangqi_ai.cpp:444`

Toggle the same fingerprint twice to restore it

XOR acts as a toggle; applying it twice cancels it, and order does not matter.

```
H ^= X;    // Add event X
H ^= X;    // Toggle again to remove X

A ^ B ^ C == C ^ A ^ B
```

$X \oplus X = 0$: the same event cancels itself.

$A \oplus B \oplus C = C \oplus A \oplus B$: combination order does not matter.

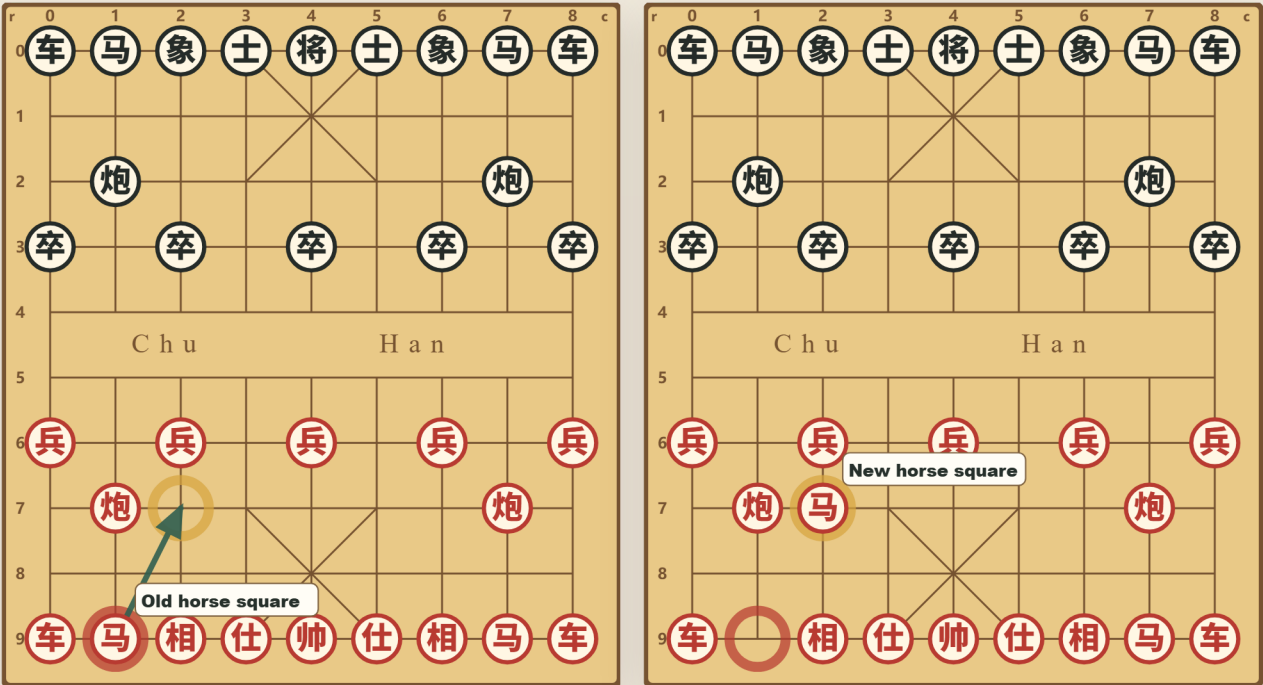
Undo repeats the same XOR operations to restore the old key.

Add, remove, and undo all use the same XOR operation.

Sources: xiangqi_ai.cpp:500 · xiangqi_ai.cpp:567

A horse move updates old square, new square, and turn

A quiet move takes three XOR operations instead of a board scan.



Before · Red to move

After · Black to move

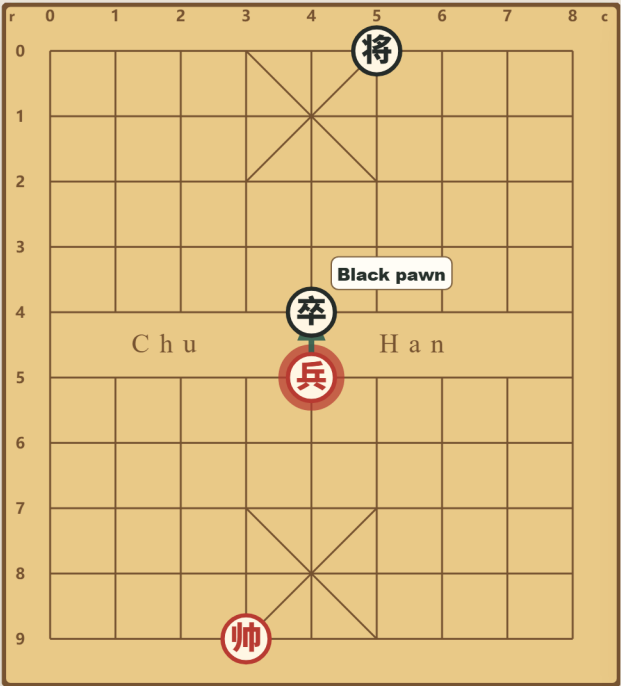
- $H \wedge= Z[\text{red horse}][9,1]$ removes the old square.
- $H \wedge= Z[\text{red horse}][7,2]$ adds the new square.
- $H \wedge= Z_TURN$ changes Red to move into Black to move.

An ordinary move replaces a board scan with three XORs.

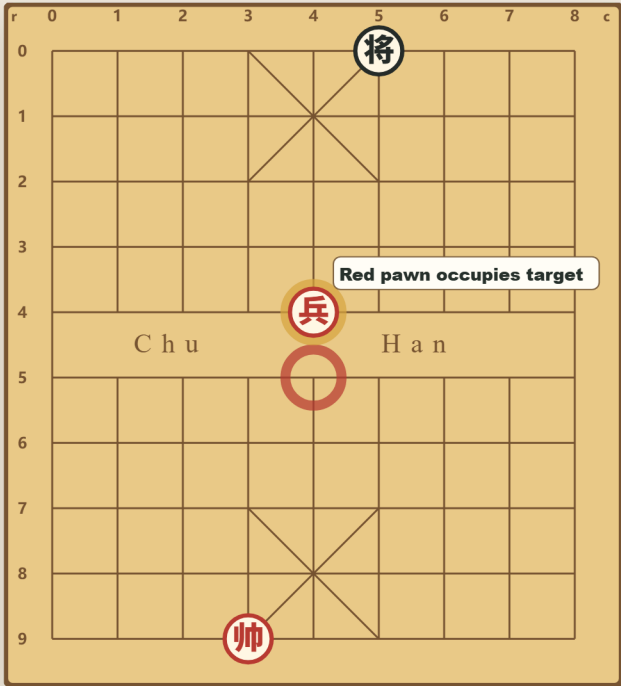
Sources: xiangqi_ai.cpp:485 · xiangqi_ai.cpp:500

A pawn capture also removes the victim

Four changes: mover source, captured target piece, mover destination, and turn.



Before · Red to move



After · Black to move

```
H ^= Z[red pawn][(5,4)]; H ^= Z[black pawn][(4,4)]
H ^= Z[red pawn][(4,4)]
H ^= Z_TURN
```

Captures also update only changed events.

Sources: xiangqi_ai.cpp:500 · xiangqi_ai.cpp:552

Every node now has a cheaply updated key

current_hash summarizes the position in 64 bits; make/undo update only changed events.

Complete state

Piece-square events
and side to move
determine
current_hash.

Incremental maintenance

The same XOR toggles
update and restore the
fingerprint.

Next question

After locating a record,
when is its search
evidence reusable?

current_hash summarizes piece placement and turn.

Make and undo update only changed events.

Next, map the key into a fixed-capacity transposition table.

Zobrist identifies positions; the TT stores search evidence.

Sources: xiangqi_ai.cpp:198 · xiangqi_ai.cpp:444 · xiangqi_ai.cpp:500

Map the key to a fixed cache slot

Different paths can reach the same position; save previous search results for reuse.

```
TT_SIZE = 1 << 23          // 2^23 slots
TT_MASK = TT_SIZE - 1       // Low 23 bits set
index = current_hash & TT_MASK
entry = transposition_table[index]
```

Why an array?

Every node probes it; direct indexing has fixed cost.

Why a mask?

Power-of-two capacity lets hash & (SIZE-1) select the low 23 bits.

What is stored?

One position-search record per slot; fields follow next.

Avoid searching the same position's subtree from scratch.

$TT_SIZE = 2^{23}$ gives indices 0 through $2^{23}-1$.

$TT_MASK = 2^{23}-1$ retains the hash's low 23 bits.

$current_hash \& TT_MASK$ selects one slot.

A mask maps the 64-bit fingerprint into a fixed-size array.

A cache record must explain its evidence

The index locates a slot; verify its full hash before deciding whether its result applies.

Field	Question answered
hash	Does the slot belong to this 64-bit position key?
depth	How deeply was this node searched?
score + flag	Is the score exact, an upper bound, or a lower bound?
best_move	Which move was most promising last time?
age	Which root-search age produced this record?

Replacing a slot

Replace if empty, same hash, old age, or the new record is at least as deep.

Starting a new search

Increment tt_age so older root-search records can be replaced sooner.

hash checks identity; depth and flag describe the score evidence.

best_move can improve ordering even when the score cannot be reused.

age and depth guide replacement under fixed capacity.

TT entries store evidence and the metadata needed to assess it.

Does the cache store a value or a bound?

Save the original entry window; reuse sufficiently deep bounds only when they cross the current threshold.

TT_EXACT

Original window [3,8],
exact result 5: return 5
directly.

TT_ALPHA: upper bound

Stored $2 \leq \text{current}$
 $\alpha=3$: this branch
cannot reach MAX's
threshold; stop.

TT_BETA: lower bound

Stored $9 \geq \text{current}$
 $\beta=8$: MIN has a
lower alternative; stop.

Insufficient depth

Do not substitute a
shallow score for
deeper search; still try
`best_move` first.

Return exact values directly.

An upper bound cuts off only when it is no greater than α .

A lower bound cuts off only when it is no less than β .

If depth or bounds are insufficient, `best_move` still improves ordering.

Use bounds as bounds; a shallow best move can still guide deeper search.

A second use for hashing

Fingerprints also detect repeated positions

Store keys along the current search path to find whether the position occurred earlier.

Current identity

current_hash
represents piece
placement and side to
move.

Path history

Save the new position's
fingerprint after each
move.

Repetition

A matching earlier key
identifies a return to the
same state.

After make, append the new key to the current path.

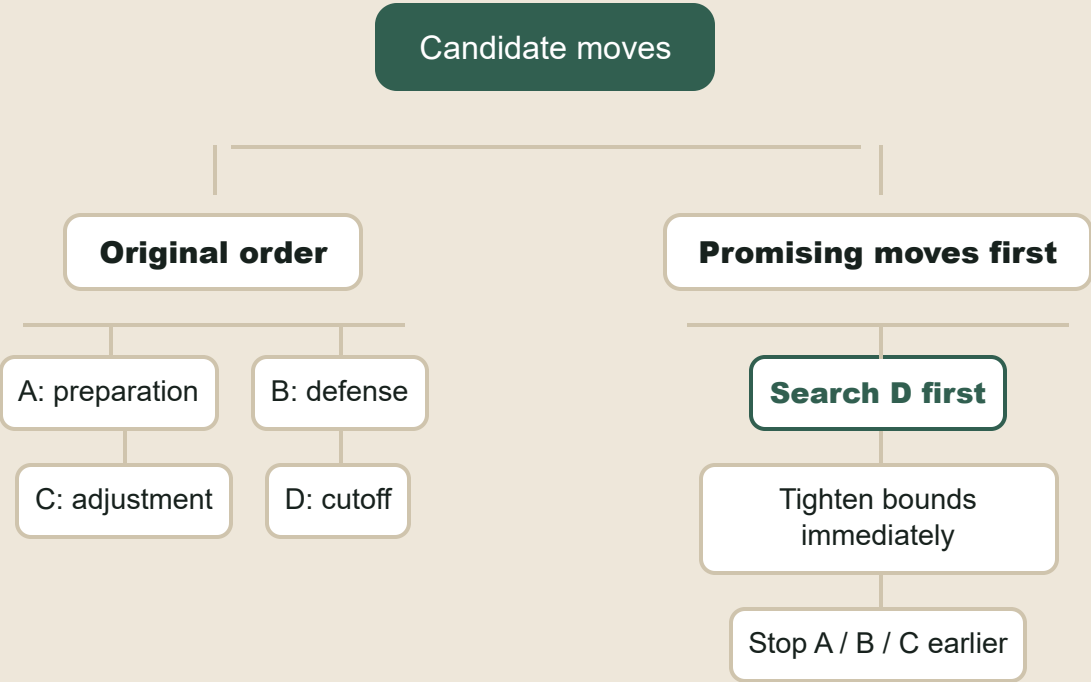
At each node, look for a matching earlier state.

The key locates repetitions; adjudication also needs the move history.

One fingerprint supports both cache lookup and path repetition detection.

Quiet moves capture nothing. Which comes first?

Quiet moves are non-captures, so victim value cannot order them.



All candidates are legal

Ordering changes recursion order without removing moves.

Cutoff move last

Three subtrees are explored before alpha-beta gets a strong bound.

Cutoff move first

The same tree establishes sooner that other candidates need no further work.

This project calls non-captures quiet moves.

Quiet moves have no captured-piece value to compare.

If D is last, we have already paid for A/B/C subtrees.

Moving D earlier preserves Minimax's result and activates alpha-beta sooner.

Ordering tests likely bound-improving moves earlier.

Three memory tables: ply, reply, and history

All three update only when a quiet move actually causes an alpha-beta cutoff.

Two killer slots

New move enters slot 0; old slot 0 becomes slot 1. No grouping by position or piece type.

Counter key

All four previous-move coordinates, not just the destination.

History limits

Depth² rewards and penalties, capped at $\pm 2^{20}$, without periodic decay.

Heuristic	Lookup key	Stored information
killer	killer_moves[ply][0 / 1]	Two recent successful quiet moves at the same search ply
counter	counter_move[previous source][previous destination]	A quiet move that cut off after that complete previous move
history	history_table[current source][current destination]	Accumulated cutoff experience for that source/destination pair

```
bonus = depth * depth;
history[successful_quiet] += bonus;
for (quiet : earlier_searched_quiets)
    history[quiet] -= bonus;
history = clamp(history, -(1<<20), +(1<<20));
```

Killer stores two moves per ply, shared across positions at that ply.

Counter indexes a reply by the previous move's four coordinates.

History accumulates positive and negative experience for candidate coordinates.

Captures do not update these tables even when they cause cutoffs.

The tables remember success at this ply, after this reply, and across long-term move history.

How do five candidates become an ordered queue?

These numbers are ordering keys, not evaluations or substitutes for search.

Example queue

A good capture → B killer → C counter → D history → E history

Multiple signals

An else-if chain uses the first matching tier; killer, counter, and history do not add together.

Captures vs experience

Captures take their own branch; killer/counter/history order quiet moves.

Candidate	Signal	Source ordering score
A: cannon takes rook, SEE nonnegative	Good capture	$10,000,000 + 1000 \times 10 - 450 = 10,009,550$
B: quiet move	Killer slot 0	9,000,000
C: quiet move	counter	7,000,000
D: quiet move	history	12,500
E: quiet move	history	-3,000

```
if (move == tt_move)      score = 300000000;
else if (good_capture)    score = 10000000 + victim*10 - attacker;
else if (move == killer1) score = 9000000;
else if (move == killer2) score = 8000000;
else if (move == counter) score = 7000000;
else                      score = history[from][to];
```

A TT best move receives 300 million and goes first.

Good captures use the 10-million tier, refined by victim and attacker values.

Killer slots use 9/8 million; counter uses 7 million.

Remaining quiet moves use descending history scores.

Ordering keys determine trial order: captures establish bounds, then experience orders quiet moves.

We can reason through replies. Now search deeper

Minimax models opponents; alpha-beta uses bounds; QS stabilizes leaves; TT reuses evidence; ordering and iterative deepening meet deadlines.

Completed

Search logic, bound-based cutoffs, tactical leaves, caching, ordering, and timely answers

Next question

Even with good ordering, there are too many moves. Which can we reduce or skip?

Risk

The next chapter moves from provable bounds to heuristic choices, with missed-move risk.

We can model opponents, use bounds, stabilize leaves, reuse memory, and return on time.

Search depth remains limited.

Next: find guide moves, use cheap probes, and prune heuristically.

Build a reliable search structure before trading heuristic risk for depth.

Sources: xiangqi_ai.cpp:1123 · xiangqi_ai.cpp:1030 · xiangqi_ai.cpp:1156

What can shallow search teach deeper search?

The previous iteration leaves a move, a score, and a likely principal variation to guide the next.

No guide move at this node

Find one cheaply before searching poor moves first.

Candidates already ordered

Ask whether later moves can challenge the current best.

Starting a new depth

Guess that the score remains near the previous iteration's result.

No guide move: first ask which move to search first.

Ordered candidates: ask whether they beat the current best.

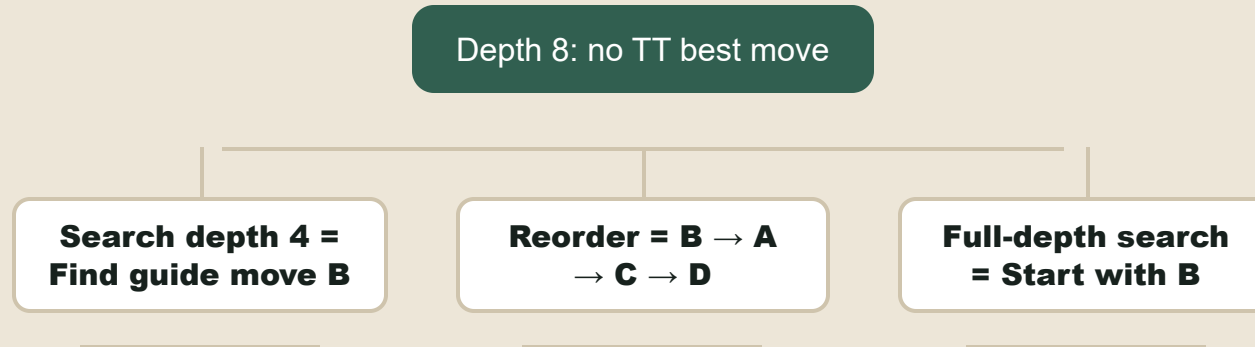
New depth: ask whether the score is still nearby.

Identify the question before naming the technique.

Sources: xiangqi_ai.cpp:1241 · xiangqi_ai.cpp:1411 · xiangqi_ai.cpp:1554

Without a guide, search shallowly to find one

A deep node with no TT move may search poor moves first and delay useful alpha-beta bounds.



```
if (!tt_move.is_valid() && depth >= 6) {  
    auto iid = shallow_search(depth - 4, alpha, beta);  
    tt_move = iid.move;        // For ordering  
}
```

At depth 8, there is no TT guide move.

Search depth 4 and retain only best move B.

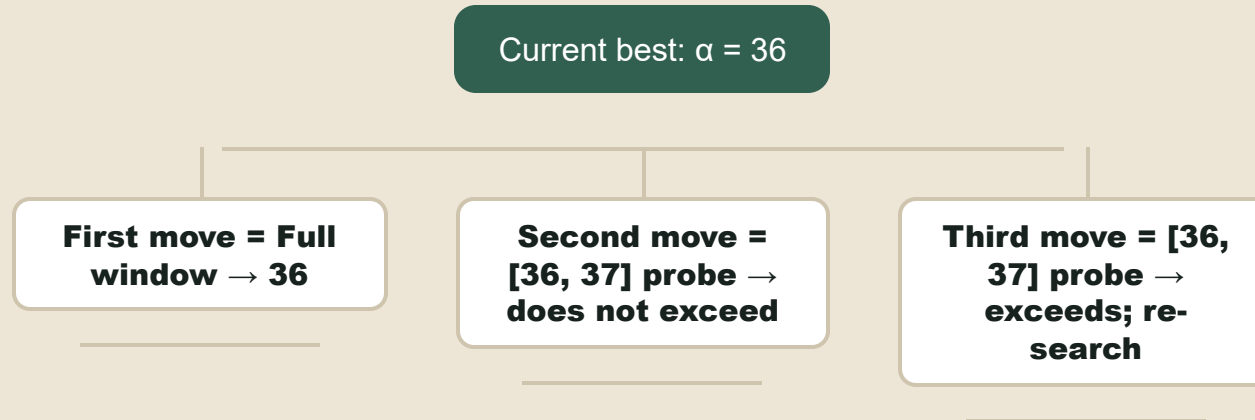
Move B to the front, then search at the target depth.

This is internal iterative deepening (IID).

IID produces a guide move, not the final score.

Later moves challenge the current best

The first ordered move establishes a principal variation; later moves use an integer null window to challenge it.



```
first move: search(alpha, beta)
later move: search(alpha, alpha + 1)
if (score > alpha && score < beta)
    search(alpha, beta)    // Resolve a successful challenge
```

The first move has no reference, so use a full window.

Later moves challenge the best with a null window.

If they fail, we know they are not preferable.

If they improve without cutting off, re-search with the full window.

This is principal variation search (PVS).

PVS probes narrowly and adds precision after a successful challenge.

Aspiration window

At a new depth, guess the score stays nearby

A narrow window around the previous result enables earlier cutoffs on both sides.

Previous iteration

Depth 6: score 120
(example)

Try a small window

Depth 7: search [90, 150]

Above the upper bound

Fail-high: widen the upper bound and re-search

Still outside

Keep widening; use the full window if needed

```
delta = 30;
alpha = prev_score - delta;
beta  = prev_score + delta;
while (score <= alpha || score >= beta) {
    widen_window();
    if (delta > 1000) use_full_window();
}
```

Depth 6 returned teaching score 120.

Start the next depth with [90, 150].

A result above the upper bound means the guess was too narrow.

Widen and re-search, returning to a full window if needed.

This is an aspiration window.

Aspiration guesses the iteration's score range and widens when wrong.

Distinguishing similar techniques

Three probes, three different questions

One finds an order, one challenges a best value, one predicts a score range.

Location	Question	Cheap action	When to do more
Deep node	Which move goes first?	IID shallow search for a guide	Always follow with target-depth search
Later move at one node	Can it beat the current best?	PVS null-window probe	Full-window re-search if inside the useful interval
New root depth	Is the score near the previous one?	Narrow aspiration window	Widen after fail-low / fail-high

- IID finds a guide move.
- PVS challenges the current best.
- Aspiration predicts the new iteration's score range.

Understand a technique by the question it answers.

Searching deeper requires accepting judgment risk

Alpha-beta cuts off with sufficient bounds; the following heuristics can miss moves.

Level 1: reduce depth

Still search the candidate, initially at lower depth.

Level 2: skip a move

Judge one candidate unlikely to be worth expanding.

Level 3: skip the node

Return a bound without searching every candidate.

Reduced depth: the candidate is still searched.

Skipped move: that candidate is not expanded.

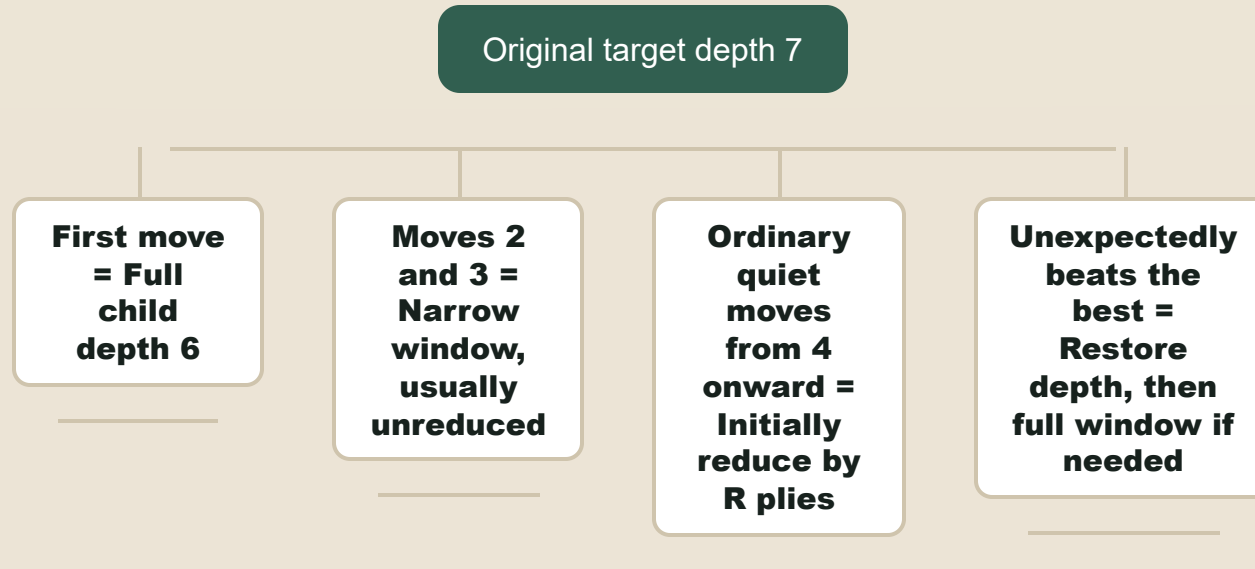
Skipped node: stop searching individual candidates.

Greater savings bring greater costs when the judgment is wrong.

Selective search trades missed-move risk for deeper focused analysis.

Reduce late quiet moves, but let them recover

At sufficient depth, reduce late non-captures that do not give check and are not killers.



```
if (late_quiet_move && !in_check && !gives_check) {  
    R = LMR_TABLE[depth][move_index]; // Initial depth reduction  
    R += history_is_bad; R -= history_is_good;  
    score = search(depth - 1 - R, narrow_window);  
    if (score challenges_best)  
        score = search(depth - 1, narrow_window);  
}
```

The first three moves establish a normal-depth reference.

Later ordinary quiet moves initially get less depth.

Good history reduces the reduction; poor history increases it.

If the reduced result challenges the best, restore full depth.

If it remains inside the useful window, re-search full-window.

LMR reduces first; underestimated candidates can recover full search.

Late move pruning (LMP)

At shallow depth, skip very late quiet moves

After many higher-priority candidates at a shallow unchecked node, skip very late quiet moves.

Conditions

Non-root; depth ≤ 8 ; not in check; non-capture; not a killer.

Count threshold

Tried candidates $> 3 + \text{depth}^2$.

Action

Skip directly to the next candidate without make or recursion.

What may be missed

A late quiet move whose preparatory value appears later.

Search earlier high-priority moves normally.

After $3 + \text{depth}^2$ candidates, skip late ordinary quiet moves.

This rule protects captures and killer moves.

Risk: a late quiet move may be strong.

LMP skips late quiet moves using order and shallow-depth conditions.

Even an optimistic margin cannot reach the threshold

The static score is too far from alpha/beta for an ordinary quiet move to change the conclusion within its margin.

Current static score

eval = 40 (example)

Shallow margin

$100 + 100 \times \text{depth}$

Current threshold

alpha = 500 (example)

Decision

$\text{eval} + \text{margin} \leq \alpha$
→ skip this ordinary quiet move

```
if (depth <= 6 && !in_check && !is_capture
    && moves_count > 1) {
    margin = 100 + 100 * depth;
    if (eval + margin <= alpha) skip_move();
}
```

The static score is far below alpha.

Allow an optimistic margin of $100 + 100 \times \text{depth}$.

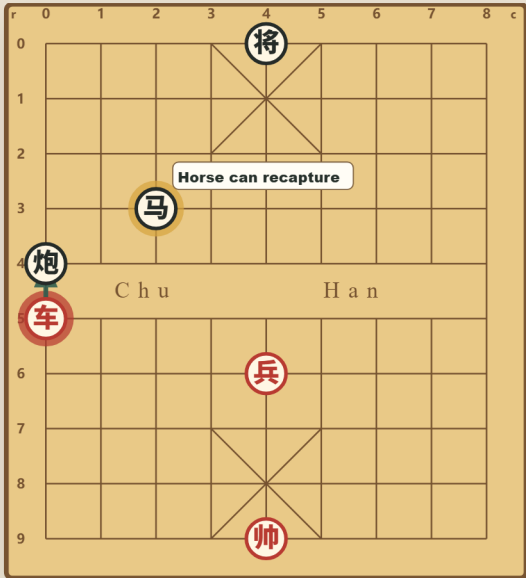
If it still cannot reach alpha, skip the move.

Risk: static evaluation can underestimate delayed gains.

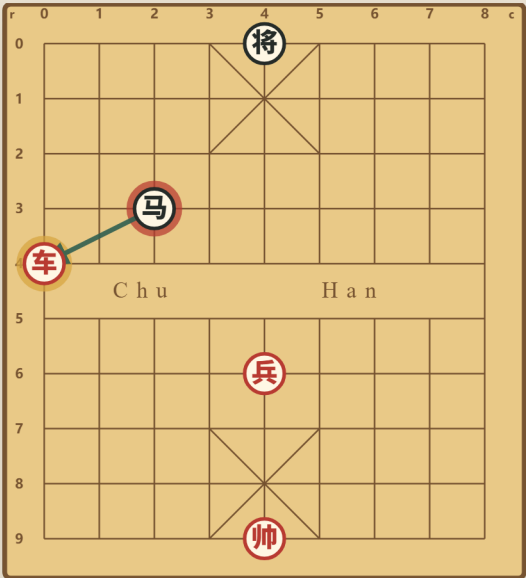
Futility uses evaluation plus a margin to judge a quiet move's prospects.

Can SEE also prune main-search candidates?

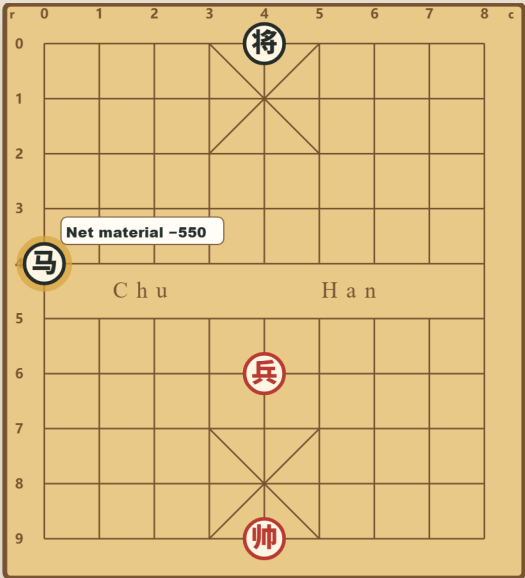
The rook-for-cannon exchange has SEE <0. Now consider the stronger action of skipping it entirely.



Red to move · Rook prepares to take the cannon at (4,0)



After rook capture · Black to move



After horse recapture · Red rook removed

At first, rook takes cannon: teaching material gain +450.

Horse (3,2) recaptures on (4,0), through empty leg (3,1).

After two plies, net change is +450 -1000 = -550.

This is a base-material ledger, not the full PST evaluate() score.

SEE supports filtering and ordering; SEE pruning skips main-search moves under restricted conditions.

An available capture may lose the exchange

At shallow unchecked nodes, skip expensive-piece captures with clearly losing target-square exchanges.

Check the shape

Depth ≤ 4 ; capture; not in check.

Compare values

Victim value < attacker value.

Simulate the exchange

SEE < -50 indicates a clear loss.

Remaining risk

Sacrifices that draw the king, open lines, or create mate may have off-square compensation.

```
if (depth <= 4 && is_capture && !in_check) {  
    if (victim_value < attacker_value && see(move) < -50)  
        skip_move();  
}
```

First compare attacker and victim values.

Then simulate alternating exchanges on the target square.

Skip clearly losing captures with SEE < -50 .

Risk: the local ledger may miss global tactical compensation.

SEE pruning filters losing captures using the target-square exchange ledger.

Can we judge an entire node?

Three questions: already good enough, apparently too poor, or still good after passing?

Already good enough?

Static evaluation minus a margin still exceeds beta.

Too far behind?

QS verifies that the position still cannot reach alpha.

Good even after a pass?

A hypothetical null move still exceeds the bound.

If static evaluation is already high, can we return a bound?

If it is very low, can QS verify it cheaply?

If passing still works, might a real move also suffice?

Shared guards: non-root, not in check, away from mate-score ranges.

Whole-node pruning saves the most and needs careful trigger conditions.

Reverse futility pruning (RFP)

Beyond the bound even after a margin

Futility asks whether one move can catch up; RFP asks whether a conservative static score can cut off the entire node.

MAX static score

eval

Conservative deduction

margin = $80 \times \text{depth}$

Still above beta

eval - margin $\geq$ beta

Return directly

Return a sufficient cutoff bound

```
if (!is_root && depth <= 7 && !in_check) {  
    margin = 80 * depth;  
    if (eval - margin >= beta)  
        return eval - margin;  
}
```

The static score starts above beta.

Subtract a conservative margin of $80 \times \text{depth}$.

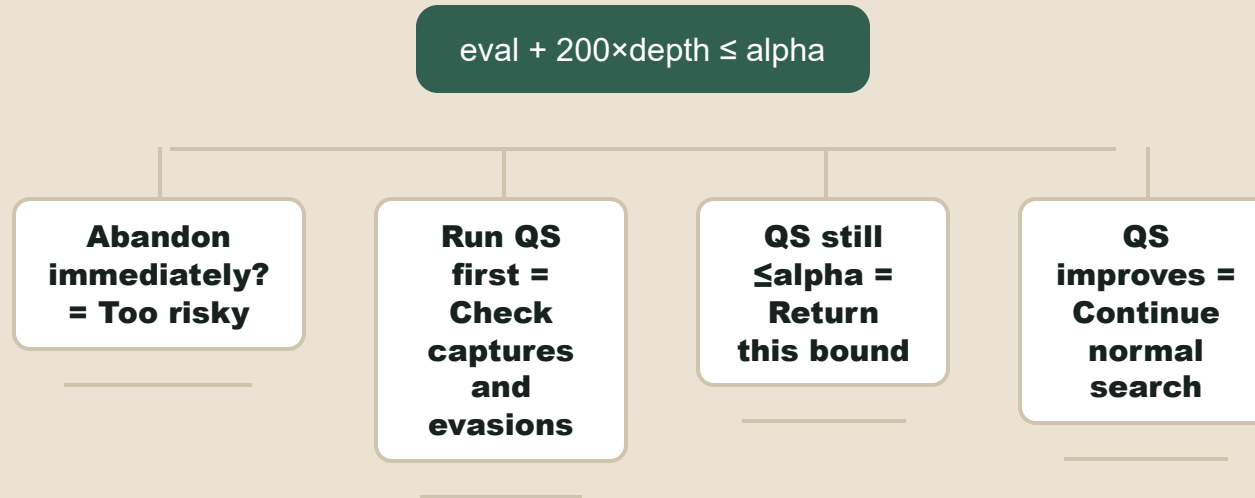
If still above, return a node-level cutoff bound.

Risk: a high static score can conceal forced counterplay.

RFP cuts off when static advantage minus a margin still clears the bound.

A very poor static score gets a tactical check

At a shallow node far below alpha, use QS first; return early only if immediate tactics cannot recover.



```
if (depth <= 3 && !in_check
    && eval + 200 * depth <= alpha) {
    q = quiescence(alpha, beta);
    if (q <= alpha) return q;
}
```

Static evaluation plus a margin remains far below alpha.

Use QS to check immediate captures and evasions.

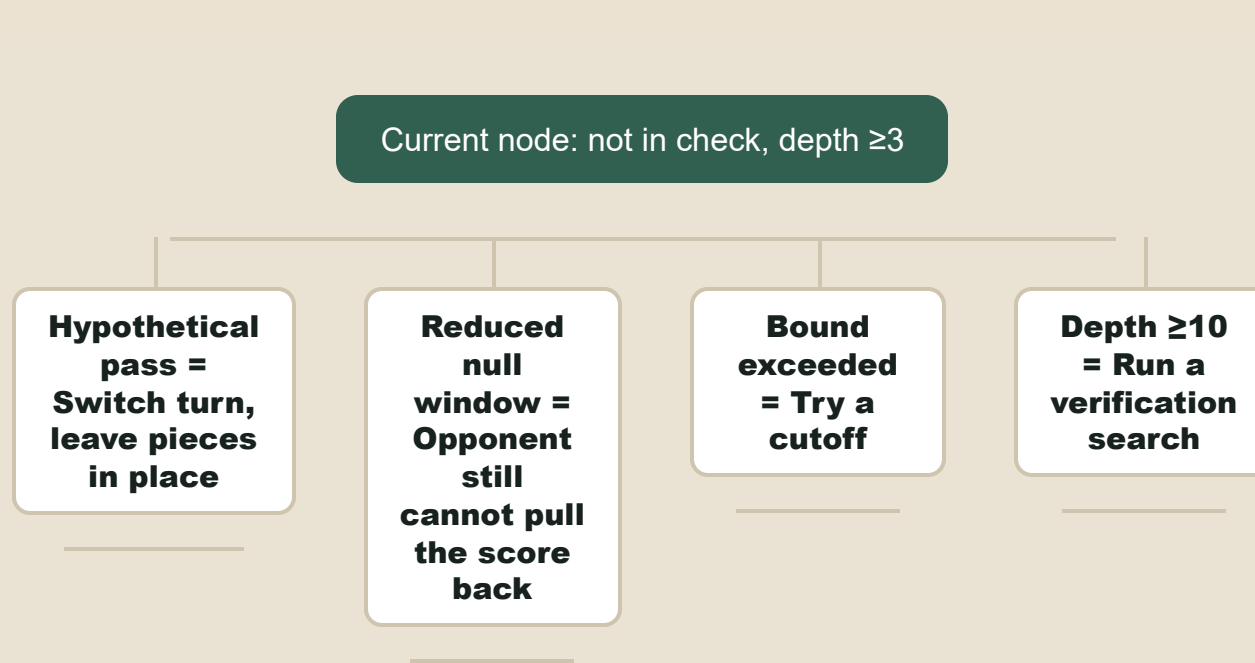
If QS cannot exceed alpha, return early.

If QS improves, resume normal main search.

Razoring verifies a poor static position with QS before returning.

Is the position good even if we pass?

Temporarily switch turn and hash without moving a piece, run a reduced null-window search, then undo fully.



```
make_null_move();           // Search-only hypothesis
R = reduced_plies(depth, eval); // Extra depth reduction
score = search(depth - 1 - R, narrow_window,
               /* allow_null = */ false);
undo_null_move();
if (cutoff && depth >= 10)
    verify_without_another_null();
```

Keep pieces fixed; switch only turn and its hash contribution.

Use only at non-root unchecked nodes with at least three plies left.

Disable another null move in the child search, preventing consecutive NMP.

At depth ≥ 10, verify a cutoff with another reduced search.

These guards reduce but do not eliminate endgame zugzwang errors.

Zugzwang challenges the null-move assumption; depth guards and verification only reduce the risk.

What if pruning misses the saving move?

The first pass allows three move-skipping rules. On either recovery trigger, restart the list with those rules disabled.

```
need_second_pass = near_losing_mate(best_score)
                    || (pruned_any && legal_searched == 0);
for (pass : {pruning_on, pruning_off_if_needed}) {
    for (move : ordered_moves_from_beginning) {
        if (pass == pruning_on)
            maybe_skip_by_LMP_Futility_SEE();
        search_normally(move); // May repeat searched moves
    }
}
```

First pass allows LMP, futility, and SEE pruning.

Trigger 1: the best score remains near a losing mate.

Trigger 2: candidates were pruned and no legal move was searched.

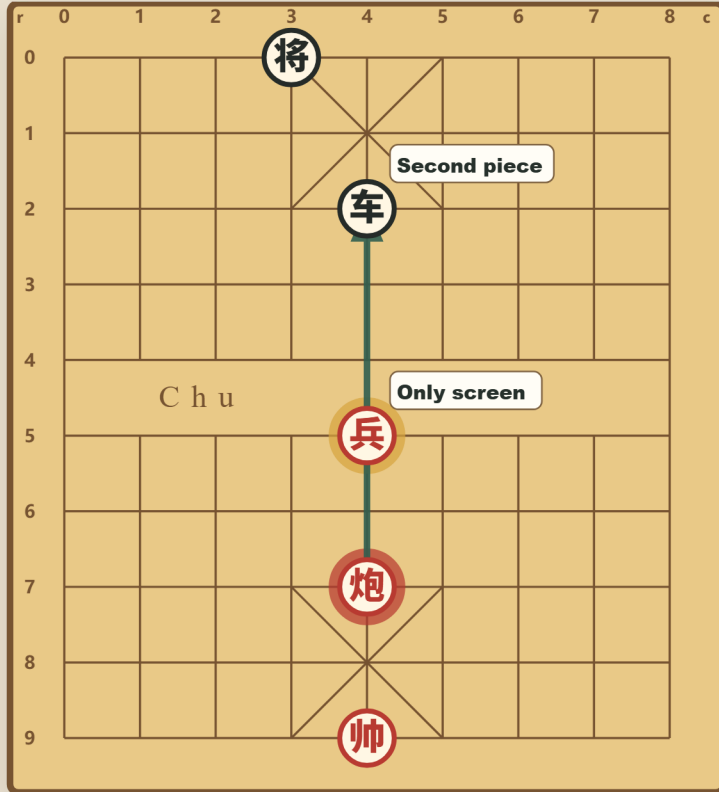
Either trigger restarts the list with all three skipping rules disabled.

Skipped moves get searched; some already searched moves are repeated.

A second pass trades repeated work for a chance to recover missed moves.

The naive approach: scan for blockers each time

For a cannon line, find the screen and then the next occupied square.



Red cannon (7,4): screen at (5,4), then black rook at (2,4)

Without precomputation, each line query must locate both blockers at runtime.

```
// Rule scan now runs in init_attack_tables
nr = sr + d;
while (inside(nr) && !occupied(nr)) {
    moves |= 1 << nr; nr += d;    // Quiet moves before screen
}
if (inside(nr)) {                // Found the screen
    nr += d;
    while (inside(nr) && !occupied(nr)) nr += d;
    if (inside(nr)) moves |= 1 << nr; // Second piece can be captured
}
```

Scan both directions from the source.

Empty squares before the screen are legal quiet destinations.

After the first piece, find the second as a capture target.

The source still uses this rule, but only to build startup tables.

The hotspot is frequency, not ten squares

The real cost is repeated line queries

A scan is short, but move generation, QS, and legality checking repeat it throughout search.

Main search

Each expanded position calls `gen_all_moves`, then `gen_moves_for` per piece.

QS

QS still generates captures or evasions at normal-search leaves.

Trial-move filtering

After each candidate, `is_in_check` queries the king's rank and file.

Repeated input

Piece type + source index on the line + current 9/10-bit occupancy.

```
minimax position
└─ gen_all_moves(...)      // Main search / QS
   └─ gen_moves_for(...)   // Each friendly piece

For every trial candidate
└─ make_move → is_in_check // Query line attacks again
```

Main search generates candidates at each expanded position.

QS continues generating tactical candidates after normal depth ends.

Legality checks query rook/cannon attacks after trial moves.

For line length L and k output moves, scanning costs $O(L)+O(k)$.

At most ten squares is cheap once, expensive across many queries.

Scan at startup; look up during search

Precompute occupancy patterns once; moves update source/target bits, then integer occupancy selects a table entry.

1. Build at startup

Ranks: 9×512 inputs; files: 10×1024 inputs. Store separate rook and cannon masks.

2. Make / undo

Update row_occ and col_occ at source/target; side_* tracks each side's occupancy.

3. Query in search

TABLE[source][occ] returns the line mask; exclude friendly pieces and enumerate set bits.

This example's file query	Integer / set bits	Meaning
col_occ[4]	676 = {2,5,7,9}	Black rook, screen, red cannon, and red king occupy the file
CANNON_COL_ATT[7][676]	324 = {2,6,8}	Capture (2,4); quiet moves to (6,4) and (8,4)
After excluding friendly pieces	{2,6,8}	All three targets remain

```
// Startup: all source ranks and 10-bit occupancy patterns
for (int sr=0; sr<10; ++sr) for (int occ=0; occ<1024; ++occ)
    CANNON_COL_ATT[sr][occ] = precompute(sr, occ);

// Make / undo: update source and destination bits
col_occ[c2] |= 1 << r2; col_occ[c1] &= ~(1 << r1);

// Search: look up, exclude friendly pieces, enumerate bits
att = CANNON_COL_ATT[r][col_occ[c]] & ~side_col_occ[side][c];
while (att) { nr=__builtin_ctz(att); att&=att-1; ADDM(r,c,nr,c); }
```

At startup, enumerate sources and occupancies to build four tables.

Initialization and make/undo maintain rank/file occupancy.

One array lookup returns attacks; then exclude friendly pieces.

ctz finds the lowest bit; mask&=mask-1 removes it; repeat for k moves.

Occupancy selects a precomputed attack mask instead of rescanning blockers.

Why did it choose that move?

The answer comes from a complete process that refines judgments within a time limit.

1 · Generate

Apply piece rules, then filter self-check after trial moves.

2 · Order

TT, captures, killer, counter, and history prioritize candidates.

3 · Search

Alternate choices; alpha-beta and PVS establish and challenge bounds.

4 · Stabilize leaves

QS, SEE, and bounded check extensions handle unfinished tactics.

5 · Allocate time

Reduce or skip selectively, with explicit missed-move risk.

6 · Return on time

Return the best move from the last complete iteration.

Rules determine which candidates enter search.

Ordering lets promising moves establish bounds early.

Evaluation guides search; concrete replies refine static preferences.

Selective search focuses the budget while accepting missed-move risk.

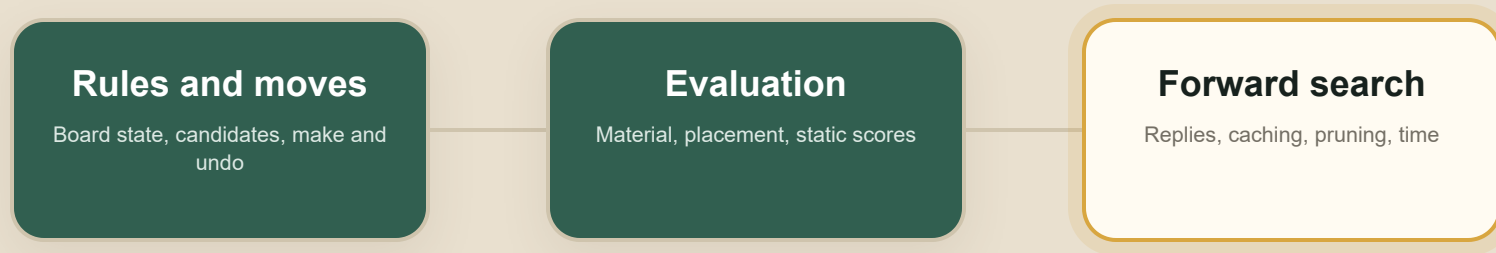
At the deadline, return the last complete iteration.

A move results from rules, memory, evaluation, and search working together.

Return to the three stages

From knowing nothing to a thinking Xiangqi system

Each practical problem adds a new capability.



Move: generate candidates and filter self-check.

Evaluate: turn material and placement into comparable static signals.

Search: consider replies and allocate time using memory, bounds, and heuristics.

A Xiangqi engine organizes rules, memory, evaluation, and search into one system.